

CSC Rahti 2 -käsikirja

Sovellusten julkaisu CSC:n konttipilveen — selaimesta, komentoriviltä ja agentilla

1. Aloitus: pääsy, kirjautuminen ja projektit
2. Sovelluksen julkaisu Dockerilla
3. GitHub-integraatio (BuildConfig ja webhookit)
4. Reitit, verkko ja URL-osoitteet
5. Ympäristömuuttujat ja salaisuudet
6. Frontend ja backend Rahdissa
7. PostgreSQL ja pgvector Rahdissa
8. Allas-objektitallennus (S3)
9. Vianmääritys
10. Agenttinen kehitys: skillit Rahti-työhön

Yhteisöohje, ei CSC:n virallinen dokumentaatio. Ajantasainen versio ja agenttiskillit:

<https://github.com/laguagu/csc-rahti-guide>

Koostettu 2026-08-31

1. Aloitus: pääsy, kirjautuminen ja projektit

Mitä Rahti on, miten sinne saa pääsyn, miten kirjaudut selaimella ja komentoriviltä, ja mitä resursseja projektisi saa käyttää.

Sisällys

- Mikä Rahti 2 on
 - Entä jos Rahti ei ole oikea paikka?
 - CSC:n muut palvelut samaan projektiin
- Edellytykset
- Kirjautuminen selaimella
- Projektin luonti
- `oc` -komentorivityökalu
- Kirjautuminen komentoriviltä
- Service account pitkäkestoiseen käyttöön
- Kiintiöt ja resurssirajat
- Tietoturvarajoitukset

Mikä Rahti 2 on

Rahti 2 on CSC:n konttipilvi. Se pyörii **OKD:llä**, joka on Red Hat OpenShiftin avoin yhteisöversio. Käytännössä: kirjoitat Dockerfilen, työnnät imagen rekisteriin ja Rahti ajaa sen podina, antaa sille julkisen HTTPS-osoitteen ja käynnistää sen uudelleen jos se kaatuu.

Rahti sopii	Rahti ei sovi
Web-sovellukset ja API:t	GPU-laskenta ja mallien koulutus → Roihu tai LUMI
Jatkuvasti päällä olevat palvelut	Eräajot ja Slurm-jobit → Roihu
Mikropalvelut ja taustapalvelut	Virtuaalikoneiden hallinta → cPouta
Demot ja opetuskäyttö	Root-oikeuksia vaativat kontit (ei mahdollista, ks. alla)

Entä jos Rahti ei ole oikea paikka?

Rahti on ilmainen CSC-projektille ja pitää datan Suomessa, mikä tekee siitä opetuskäyttöön vaikeasti voitettavan. Kaikkein se ei silti ole nopein reitti:

Alusta	Milloin parempi kuin Rahti	Huomioi
Vercel	Next.js/React-frontend, esikatselu-URLit jokaisesta PR:stä, reunatoiminnot	Ilmaistaso ei salli kaupallista käyttöä; data USA:ssa oletuksena
Render	Taustapalvelu + hallittu Postgres ilman Kubernetesia	Ilmaistason palvelut nukahtavat käyttämättöminä
Railway	Nopein "repo sisään, URL ulos" kokeiluihin	Käyttöperusteinen hinnoittelu yllättää helposti
Hetzner tai cPouta	Tarvitset itse virtuaalikoneita, GPU:ta tai root-oikeuksia	Ylläpito, päivitykset ja tietoturva ovat sinun

Käytännössä sama Dockerfile toimii kaikissa näissä — ero on siinä, kuka hoitaa konfiguraation. Rahdissa kirjoitat Kubernetes-objektit itse; Vercelissä ja Railwayssa alusta arvaa ne puolestasi. Opetusmielessä Rahti opettaa enemmän, koska objektit ovat näkyvissä.

Opiskelijaprojektissa tavallinen yhdistelmä on frontend Vercelissä ja raskaampi backend + tietokanta Rahdissa: julkinen demo-URL syntyy helposti, mutta data ja mallit pysyvät CSC:n puolella.

Keskeiset osoitteet:

Mitä	Osoite
Web-konsoli	<code>https://console.rahti.csc.fi</code>
API-palvelin	<code>https://api.2.rahti.csc.fi:6443</code>
Sisäinen konttirekisteri	<code>image-registry.apps.2.rahti.csc.fi</code>
Sovellusten URL-avaruus	<code>*.2.rahtiapp.fi</code>
Ulosmenevä IP (palomuurisääntöihin)	<code>86.50.229.150</code> — voi muuttua, tarkista aina dokumentaatiosta

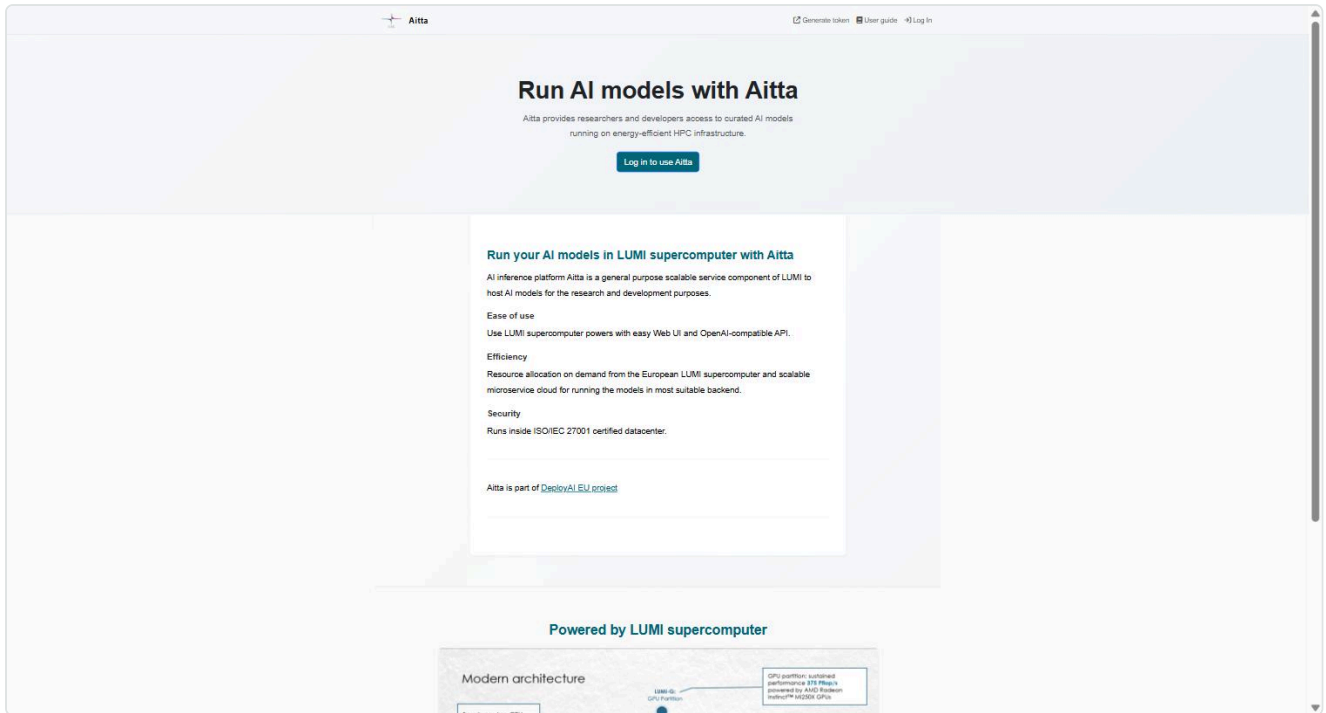
CSC:n muut palvelut samaan projektiin

Rahti-oikeus avaa oven muihinkin CSC:n palveluihin samalla laskentaprojektilla. Näitä tarvitaan usein samassa sovelluksessa:

Palvelu	Mihin	Ohje
Satama	Konttirekisteri (Harbor), imagejen jako projektien välillä, haavoittuvuusskannaus	luku 2
Allas	Objektitallennus S3-rajapinnalla, isot tiedostot ja varmuuskopiot	luku 8
Aitta	Valmiit kieli- ja embeddings-mallit OpenAI-yhteensopivan API:n kautta	csc-rahti-skilli
Roihu	NVIDIA GH200 -GPU:t, mallien koulutus ja eräajot	csc-roihu-skilli
LUMI	AMD MI250X, EuroHPC-kiintiö	csc-lumi-skilli

Aitta on erityisen kätevä opetuskäytössä: saat kieli- ja embeddings-mallit rajapinnan takaa ilman omaa GPU:ta ja ilman kaupallista API-avainta. Mallit ajetaan LUMI-supertietokoneella ja rajapinta on

OpenAI-yhteensopiva, joten olemassa oleva koodi toimii vaihtamalla `baseURL` .



```
from openai import OpenAI

client = OpenAI(
    base_url="https://aitta-api.csc.fi/openai",    # AITTA_API_BASE
    api_key="<AITTA_API_TOKEN>",                # token: https://aitta-auth.csc.fi
)
```

Mallivalikoimassa on mm. Llama 3.3, gpt-oss, Gemma 3, Qwen3-VL, embeddings-malleja ja suomalainen **Poro 2**. Useimmat mallit ovat "offline"-tilassa, eli ne pitää herättää kutsulla `POST /model/{id}/preload` ennen ensimmäistä chat-kutsua.

Edellytykset

1. **CSC-käyttäjätunnus** — luodaan [MyCSC](#)-palvelussa.
2. **Laskentaprojekti, jolle Rahti on otettu käyttöön:**
 - Kirjaudu [my.csc.fi](#) → *My Projects*
 - Valitse projekti → avaa palvelulistasta **Rahti** → hyväksy käyttöehdot → *Apply for access*
 - CSC vahvistaa hakemuksen. Oikeuksien synkronointiin voi mennä ~10 minuuttia.
3. **Docker tai Podman** paikallisesti, jos rakennat imaget itse.
4. **oc -komentorivityökalu**, jos et halua tehdä kaikkea selaimessa.

Työkalujen asennus lyhyesti:

Työkalu	Windows	macOS	Linux
Docker	Docker Desktop tai winget install Docker.DockerDesktop	Docker Desktop tai brew install --cask docker	sudo apt install docker.io + sudo usermod -aG docker \$USER
Podman (kevyempi vaihtoehto)	winget install RedHat.Podman	brew install podman	sudo apt install podman
oc	scoop install openshift- cli	brew install openshift-cli	lataa konsolista

Tarkista lopuksi että molemmat vastaavat:

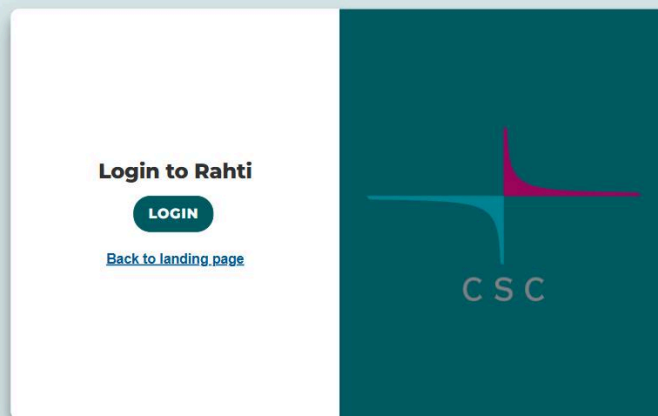
```
docker --version      # tai: podman --version
oc version --client
```

Podman ajaa kontit ilman root-oikeuksia ja on siksi lähempänä sitä, miten Rahti ajaa ne. Komennot ovat samat: korvaa `docker` sanalla `podman`.

Sama laskentaprojekti voi olla käytössä myös cPoutassa tai Roihussa — Rahti lisätään vain palveluvalikoimaan.

Kirjautuminen selaimella

Mene osoitteeseen <https://console.rahti.csc.fi> ja paina **LOGIN**.



Valitse tunnistautumistapa: **Haka** (korkeakoulut), **Virtu** (valtionhallinto) tai **CSC**. Kaikki vaativat monivaiheisen tunnistautumisen (MFA) — pakollinen 25.11.2025 alkaen. MFA:ta ei voi ohittaa skriptillä eikä tekoälyagentilla.

"User not found" kirjautumisen jälkeen? Tunnus on olemassa, mutta laskentaprojektille ei ole vielä myönnetty Rahti-oikeutta, tai synkronointi on kesken. Tarkista MyCSC:stä.

Projektin luonti

Rahdissa kaikki ajetaan **projektin** (Kubernetes-termein *namespace*) sisällä. Projektilla on oma verkko, omat salaisuudet ja oma käyttöoikeuslista.

Selaimessa: vasemmasta valikosta *Home* → *Projects* → **Create Project**.

Komentoriviltä — huomaa `csc_project` -kuvaus, joka kytkee Rahti-projektin laskentaprojektiin ja siten laskutukseen:

```
oc new-project minun-projekti --description="csc_project: 2001234"
```

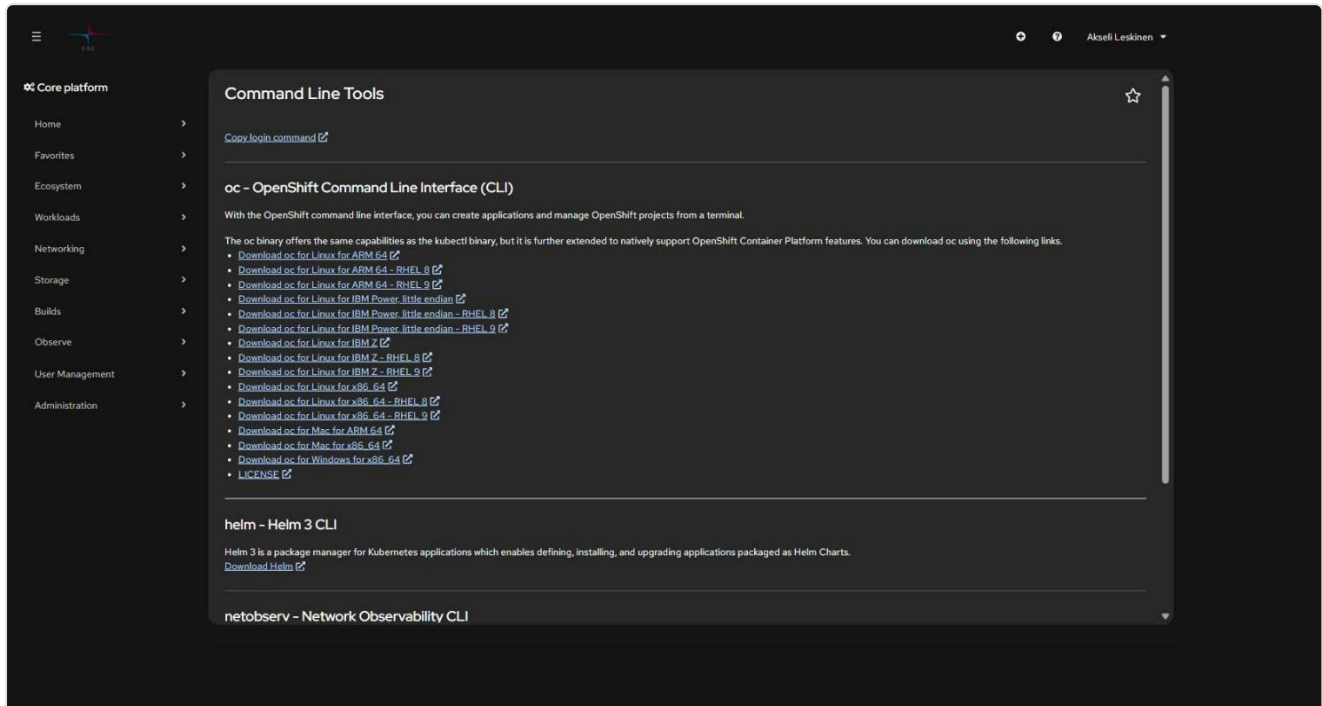
Oletuksena kaikki laskentaprojektin jäsenet saavat admin-oikeudet luotuun Rahti-projektiin. Yksittäisiä käyttäjiä voi lisätä konsolissa *User Management* → *RoleBindings*.

Nimeäminen: projektin nimi näkyy sovelluksen oletusosoitteessa muodossa `<sovellus>-<projekti>.2.rahtiapp.fi`, joten valitse lyhyt ja selkeä nimi.

oc -komentorivityökalu

oc on OpenShiftin CLI. Se osaa kaiken minkä `kubectl`, ja lisäksi OKD-spesifit asiat (routet, imagestreamit, buildconfigit).

Lataa binääri suoraan konsolista: **?-valikko** → **Command Line Tools**.



Windowsilla helpoin tapa on paketinhallinta:

```
# Scoop
scoop install openshift-cli

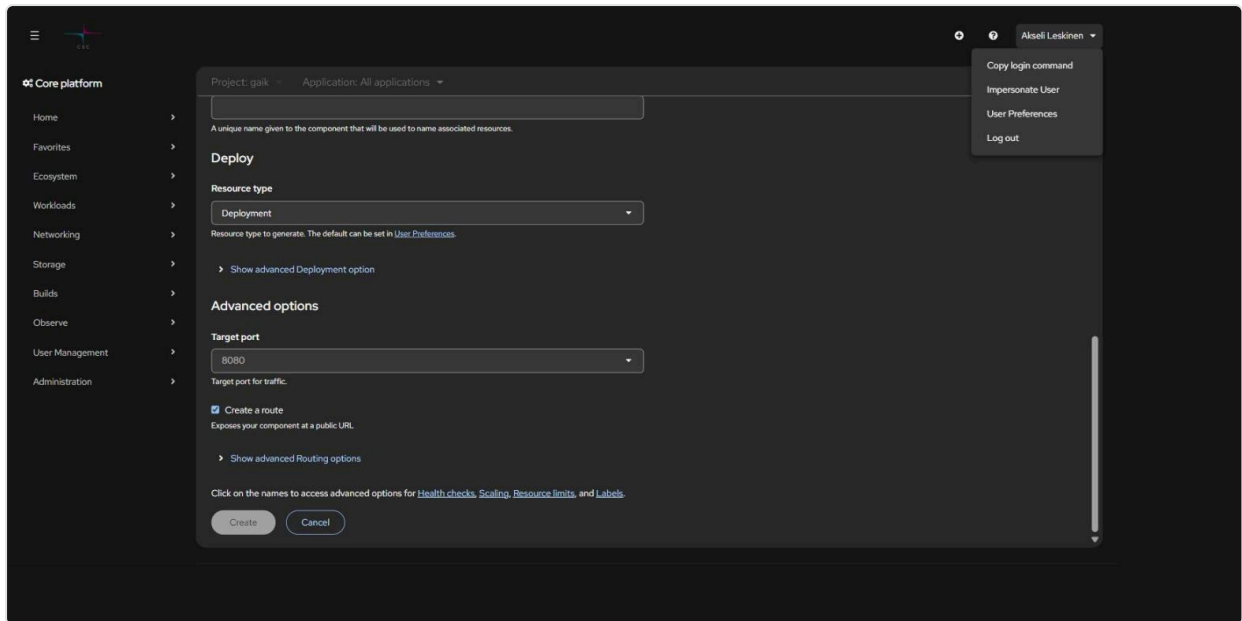
# tai lataa zip yllä olevalta sivulta ja pura oc.exe PATHissa olevaan kansioon
```

Tarkista asennus:

```
oc version --client
```

Kirjautuminen komentoriviltä

1. Klikkaa konsolissa oikeasta yläkulmasta nimeäsi → **Copy login command**



2. Paina avautuvalla sivulla **Display Token**

3. Kopioi komento ja aja se terminaalissa:

```
oc login https://api.2.rahti.csc.fi:6443 --token=sha256~<sinun-tokenisi>
```

Kirjautuminen on konekohtainen ja jaettu kaikkien terminaali-ikkunoiden kesken. **Henkilökohtainen token vanhenee noin vuorokaudessa.**

Tarkista kuka olet ja missä projektissa:

```
oc whoami           # käyttäjätunnus tai system:serviceaccount:<ns>:<nimi>
oc project           # nykyinen projekti
oc projects          # kaikki projektit joihin on pääsy
oc project minun-projekti # vaihda projektia
```

Service account pitkäkestoiseen käyttöön

Henkilökohtainen token vanhenee joka päivä, mikä on kiusallista CI-putkissa, deploy-skripteissä ja tekoälyagenteilla. Ratkaisu on **service account**: sillä ei ole MFA:ta ja sen tokenin eliniän saa itse valita.

```
# 1. Luo tunnus projektiin
oc create serviceaccount deployer-bot -n minun-projekti

# 2. Anna sille oikeudet (view = luku, edit = deployaus, admin = myös oikeuksien hallinta)
oc adm policy add-role-to-user edit \
  system:serviceaccount:minun-projekti:deployer-bot -n minun-projekti

# 3. Pyydä token - tässä vuosi
oc create token deployer-bot -n minun-projekti --duration=8760h
```

Säilytä token salaisuuksien hallinnassa tai gitignoratussa env-tiedostossa — **älä koskaan committaa sitä**. Käyttö:

```
oc login https://api.2.rahti.csc.fi:6443 --token=<service-account-token>
```

Peruutus on yhtä nopeaa:

```
oc adm policy remove-role-from-user edit \
  system:serviceaccount:minun-projekti:deployer-bot -n minun-projekti
oc delete serviceaccount deployer-bot -n minun-projekti
```

Käytä eri tunnusta eri tarkoituksiin. Pelkkään imagen työntämiseen riittää rooli `system:image-pusher`, ei `edit`.

Tämän repositorion [csc-rahti-skilli](#) sisältää PowerShell-skriptit (`Initialize-RahtiCredential.ps1`, `Connect-Rahti.ps1`), jotka automatisoivat koko elinkaaren ja tallettavat tokenin repositorion ulkopuolelle.

Kiintiöt ja resurssirajat

Kiintiö on **laskentaprojektikohtainen ja jaettu kaikkien sen Rahti-projektien kesken** — ei siis per sovellus. Oletuskiintiö uudelle laskentaprojektille:

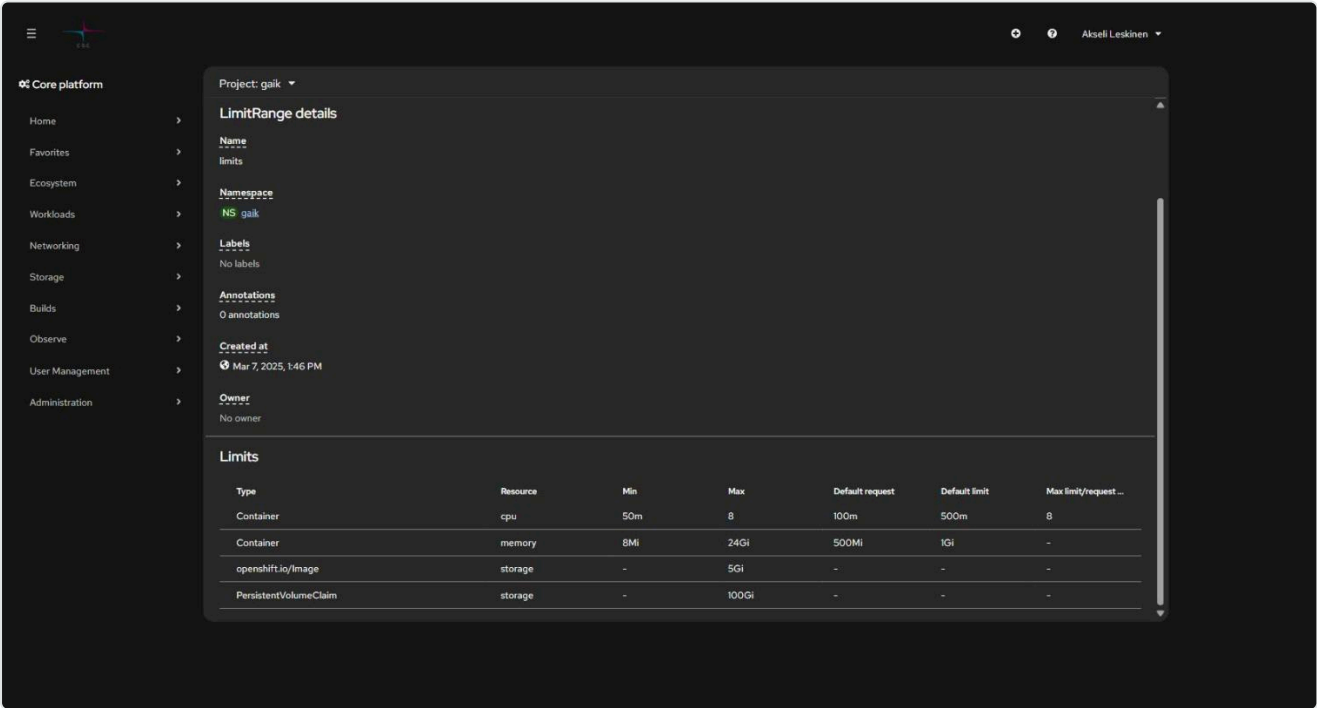
Resurssi	Oletus
Virtuaaliytimet	4
Muisti	16 GiB
Levy (PVC)	100 GiB
Väliaikaislevy (ephemeral)	5 GiB
ImageStreamien määrä	20
Podien yhtäaikainen määrä	100
PVC:iden määrä	20

```
oc describe AppliedClusterResourceQuotas # koko laskentaprojektin käyttö
oc describe limitranges -n <namespace> # yksittäisen kontin rajat
```

Oletukset osuvat podiin vaikka Deploymentissa ei pyytäisi mitään:

```
oc get pod <pod> -o jsonpath='{.spec.containers[0].resources}'
# {"limits":{"cpu":"500m","memory":"1Gi"},"requests":{"cpu":"100m","memory":"500Mi"}}
```

LimitRange määrää, mitä yksittäinen kontti saa pyytää — ja mitä se saa jos et pyydä mitään:



Tyyppi	CPU	Muisti
<code>requests</code> (varaus)	100m	500Mi
<code>limits</code> (katto)	500m	1Gi

Lisäksi: `limits / requests` -suhde saa olla korkeintaan 5, yksittäinen image enintään 5 GiB ja yksittäinen PVC enintään 100 GiB. Yllä olevassa kuvassa projektille on myönnetty oletusta suuremmat rajat — **tarkista aina oman projektisi todelliset arvot**, älä oleta.

Lisäkiintiötä haetaan [CSC:n palvelupisteestä](#) tapauskohtaisesti.

Tietoturvarajoitukset

Rahti on monen asiakkaan jaettu ympäristö, joten kontteja rajoitetaan:

- **Root ei ole sallittu.** Kontti, joka vaatii `USER root`, ei käynnisty.
- **Satunnainen UID.** Kontti saa käynnistyessään projektikohtaisen UID:n. Ohjeen testisovellus sai arvon `1006240000`, vaikka sen Dockerfile sanoo `USER 1001`. Älä siis kirjoita `runAsUser` -arvoa manifestiin äläkä oleta UID:tä. **Ryhmä on aina 0** — siihen perustuu kirjoitusoikeuksien ratkaisu.
- **Restricted-v2 -käytäntö:** `allowPrivilegeEscalation` ei saa olla `true`, kaikki capabilityt on pudotettava (`capabilities.drop: ALL`, poikkeuksena `NET_BIND_SERVICE`), ja `seccompProfile` joko tyhjä tai `RuntimeDefault`.
- **Privileged-tila on estetty.**

Käytännön seuraus Dockerfileen: tee kirjoitettavista hakemistoista ryhmän 0 kirjoitettavia. Katso [09 Vianmääritys](#) ja skillin [Dockerfile-esimerkit](#).

Seuraava: [2. Sovelluksen julkaisu](#) →

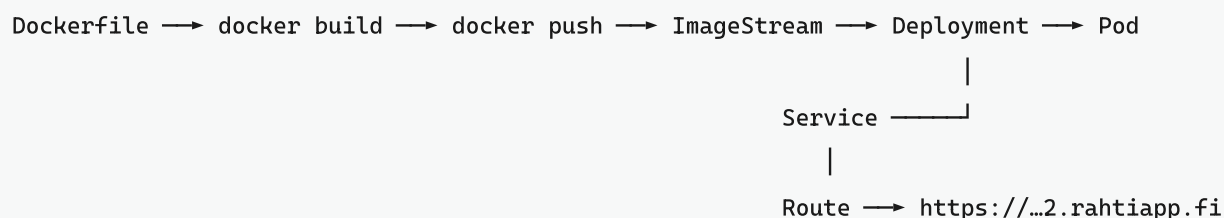
2. Sovelluksen julkaisu Dockerilla

Imagen rakennus, työntäminen rekisteriin ja sovelluksen käynnistys Rahdissa — sekä selaimesta että komentoriviltä.

Sisällys

- Kokonaiskuva
- Nopein reitti alusta loppuun
- 1. Rakenna image
- 2. Valitse rekisteri
 - Vaihtoehto A: Rahdin sisäinen rekisteri
 - Vaihtoehto B: Satama (CSC:n Harbor)
 - Vaihtoehto C: Docker Hub
- 3. Julkaise selaimesta
- 4. Julkaise komentoriviltä (manifestit)
- Automaattinen uudelleenkäynnistys uudesta imagesta
- Toistuva deploy-skripti
- Binary build vaihtoehtona

Kokonaiskuva



Neljä objektia riittää lähes aina:

Objekti	Tehtävä
ImageStream	Rahdin sisäinen viittaus imageen ja sen tageihin
Deployment	Pitää halutun määrän podeja pystyssä, hoitaa rullaavan päivityksen
Service	Pysyvä sisäinen osoite podeille (DNS-nimi + kuormantasaus)
Route	Julkinen HTTPS-osoite, joka osoittaa Serviceen

Nopein reitti alusta loppuun

Tämä komentosarja on ajettu läpi sellaisenaan Rahti 2:ssa esimerkisovelluksella `examples/hello-rahti`. Jos haluat vain nähdä jonkin toimivan ensin, kloonaa repo ja aja nämä:

```
cd examples/hello-rahti
oc project <projekti>

oc create imagestream hello-rahti # ENNEN pushia
docker build --platform linux/amd64 -t hello-rahti .
docker login -u unused -p $(oc whoami -t) image-registry.apps.2.rahti.csc.fi
docker tag hello-rahti image-registry.apps.2.rahti.csc.fi/<projekti>/hello-rahti:latest
docker push image-registry.apps.2.rahti.csc.fi/<projekti>/hello-rahti:latest

oc new-app --image-stream=hello-rahti # luo Deploymentin
oc expose deployment/hello-rahti --port=8080 # luo Servicen - EI tule
automaattisesti
oc create route edge hello-rahti --service=hello-rahti --insecure-policy=Redirect

curl -s https://$(oc get route hello-rahti -o jsonpath='{.spec.host}')/health
```

oc new-app ei luo Serviceä. Se luo pelkän Deploymentin, ja reitin luonti kaatuu virheeseen *"you need to provide a route port via --port when exposing a non-existent service"*. `oc expose deployment/<nimi> --port=<portti>` luo Servicen oikealla valitsimella. Tämä on yksi tämän ohjeen useimmin kokeilluista kohdista — ja se puuttuu useimmista netin ohjeista.

Loput tästä luvusta selittää mitä kukin vaihe tekee ja miten sen tekee kestävämmiin.

1. Rakenna image

```
# Peruskomento projektin juuressa
docker build -t sovellukseni .

# Ilman välimuistia, jos epäilet vanhentunutta kerrosta
docker build --no-cache -t sovellukseni .
```

Testaa paikallisesti ennen kuin viet pilveen:

```
docker run -p 3000:3000 sovellukseni
docker run --env-file .env -p 3000:3000 sovellukseni
```

Rahti-yhteensopiva Dockerfile: älä aja roottina, älä kovakoodaa UID:tä ja tee kirjoitettavista hakemistoista ryhmän 0 kirjoitettavia. Valmiit pohjat (Next.js, Python/FastAPI, staattinen sivusto) löytyvät skillin [dockerfile-examples.md](#)-tiedostosta.

Proessorarkkitehtuuri: Rahti ajaa `linux/amd64`. Applen M-sarjan koneella rakennettu image on oletuksena `arm64` eikä käynnisty Rahdissa. Käytä `docker build --platform linux/amd64` ...

2. Valitse rekisteri

Rekisteri	Milloin
Rahdin sisäinen <code>image-registry.apps.2.rahti.csc.fi</code>	Nopea kehityssykli yhden projektin sisällä. Oletusvalinta.
Satama <code>satama.csc.fi</code>	Image jaetaan projektien tai klustereiden välillä, tai tarvitaan haavoittuvuusskannaus, allekirjoitus ja pitkäikäiset robottitunnukset.
Docker Hub	Julkinen image, tai ei CSC-projektia mukana lainkaan.

Vaihtoehto A: Rahdin sisäinen rekisteri

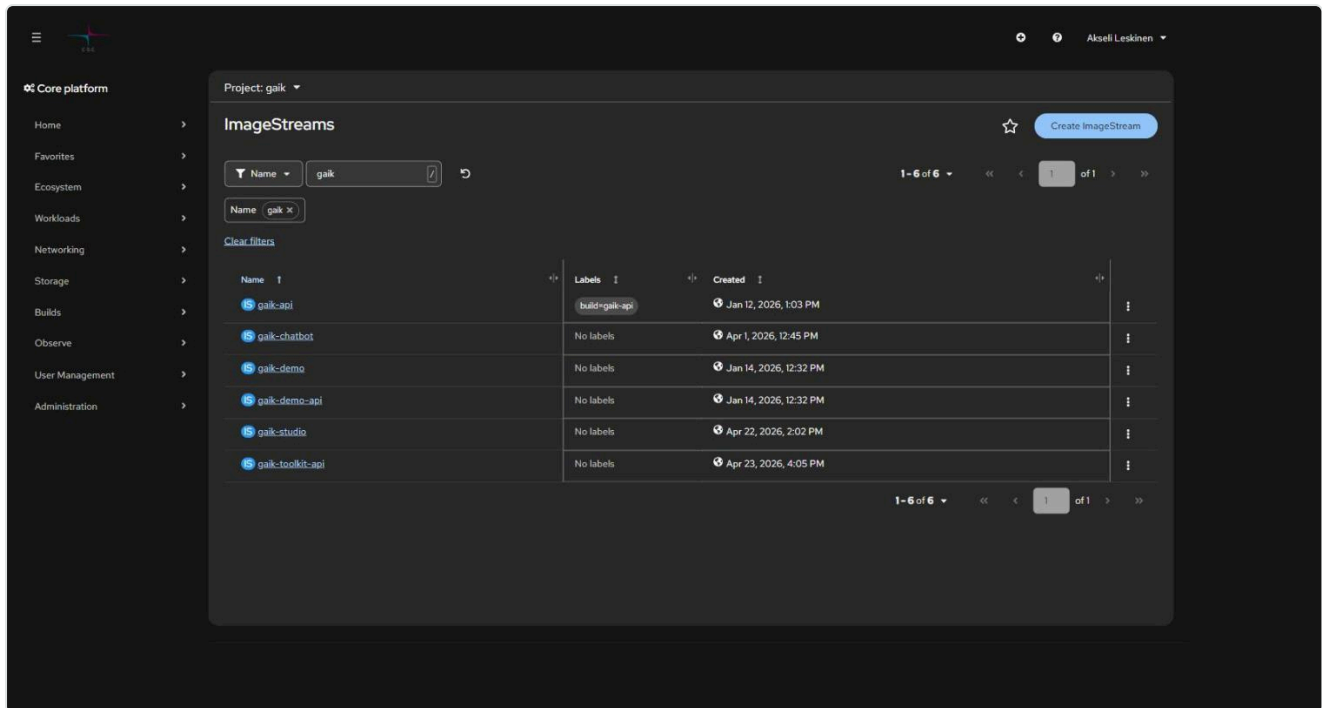
```
# 1. Luo ImageStream ENSIN - ilman sitä push kaatuu HTTP 500 -virheeseen
oc get is -n <projekti>                                # listaa olemassa olevat
oc create imagestream sovellukseni -n <projekti>

# 2. Kirjaudu rekisteriin (vaatii voimassa olevan oc-session)
docker login -u unused -p $(oc whoami -t) image-registry.apps.2.rahti.csc.fi

# 3. Tagaa
docker tag sovellukseni \
  image-registry.apps.2.rahti.csc.fi/<projekti>/sovellukseni:latest

# 4. Työnnä
docker push image-registry.apps.2.rahti.csc.fi/<projekti>/sovellukseni:latest
```

Jos ImageStream puuttuu, push päättyy virheeseen `unexpected status from HEAD request ... : 500`, joka ei mainitse ImageStreamia sanallakaan. Muista siis järjestys.



Oikotie ensimmäisellä kerralla: työnnä image ensin Docker Hubiin ja julkaise se Rahdissa sieltä. Rahti luo ImageStreamin automaattisesti, minkä jälkeen voit siirtyä sisäiseen rekisteriin.

CI-putkeen kannattaa tehdä oma tunnus, jolla on vain työntöoikeus:

```
oc create serviceaccount pusher -n <projekti>
oc policy add-role-to-user system:image-pusher -z pusher -n <projekti>
docker login -u unused -p $(oc create token pusher --duration=8760h) \
  image-registry.apps.2.rahti.csc.fi
```

Vaihtoehto B: Satama (CSC:n Harbor)

Satama on CSC:n oma konttirekisteri, erillinen palvelu Rahdista. Kirjautuminen tapahtuu **CLI-salaisuudella** (Satama-käyttöliittymä → käyttäjänimi → *User Profile* → *CLI Secret*), ei MyCSC-salasanalla.

☐	Project Name	Access Level	Role	Type	Repositories Count	Creation Time
☐	library	Public	-	Project	19	9/16/25, 9:52 AM
☐	project_2017485	Private	Project Admin	Project	0	8/3/26, 9:30 AM
☐	r_installation_aida	Public	-	Project	6	3/18/26, 7:56 AM
☐	r_installation_general	Public	-	Project	0	3/18/26, 7:56 AM
☐	r_installation_spack	Public	-	Project	6	3/18/26, 7:56 AM

Näkymässä `library` ja `r_installation_*` ovat CSC:n omia julkisia projekteja, joista saa vetää imagen ilman kirjautumista. `project_<numero>` on oma laskentaprojektisi.

```
docker login satama.csc.fi -u <csc-tunnus>      # liitä CLI secret
docker tag sovellukseni:1.0 satama.csc.fi/<satama-projekti>/sovellukseni:1.0
docker push satama.csc.fi/<satama-projekti>/sovellukseni:1.0
```

Yksityisestä Satama-projektista vetäminen vaatii pull secretin:

```
oc create secret docker-registry satama-pull \
  --docker-server=satama.csc.fi \
  --docker-username='robot$<projekti>+<nimi>' \
  --docker-password='$SATAMA_ROBOT_SECRET' -n <projekti>
oc secrets link default satama-pull --for=pull -n <projekti>
```

Julkisista Satama-projekteista vedetään ilman salaisuutta. Robottitunnukset, skannaus, tagien muuttumattomuus ja kiintiöt: satama-registry.md.

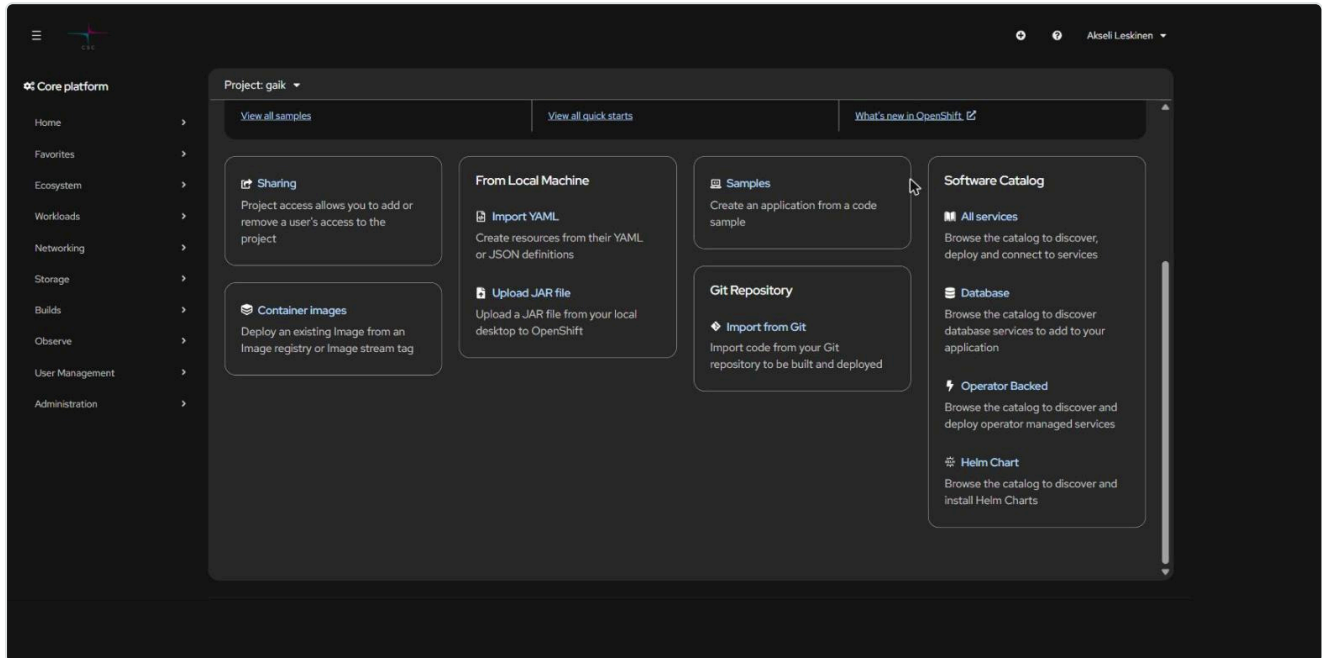
Vaihtoehto C: Docker Hub

```
docker tag sovellukseni <dockerhub-tunnus>/sovellukseni:latest
docker push <dockerhub-tunnus>/sovellukseni:latest
```

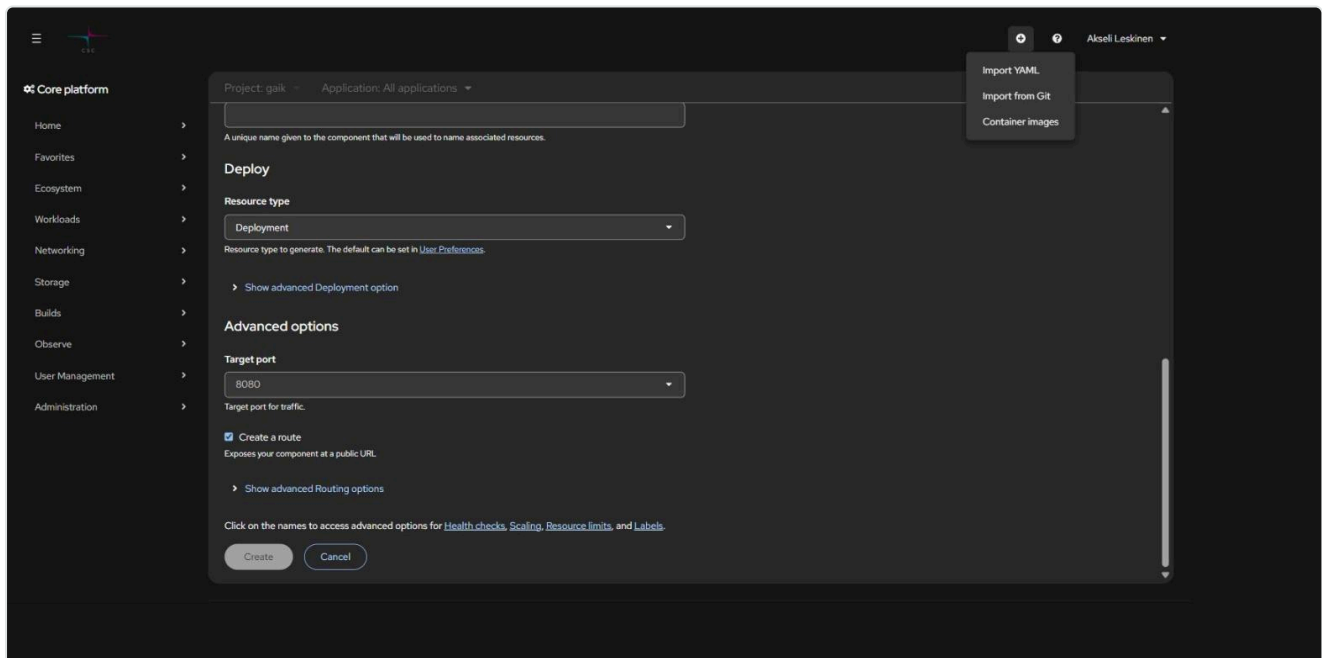
Yksityinen Docker Hub -image vaatii Rahdissa *Image pull secretin* (registry `docker.io`, käyttäjätunnus ja salasana/token). Julkiselle imagelle ei tarvita mitään.

3. Julkaise selaimesta

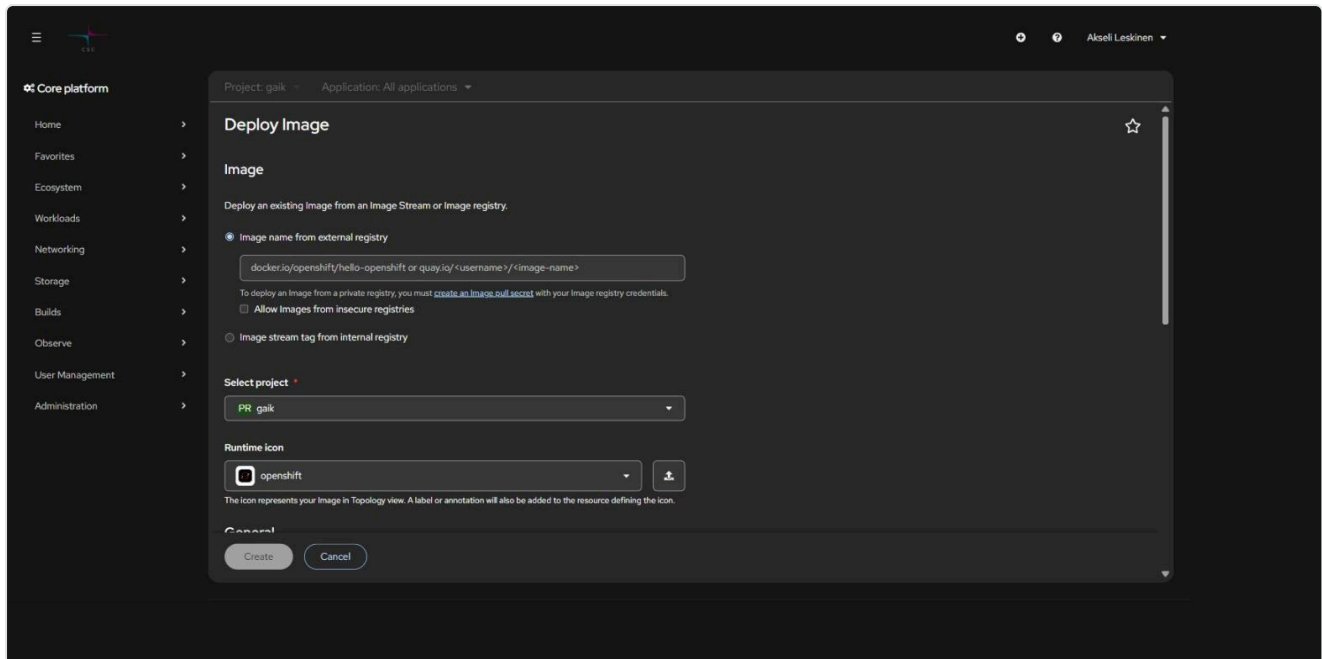
Konsolin vasen valikko on nykyään yhtenäinen **Core platform** -navigaatio; erillistä Developer/Administrator-näkymän vaihtoa ei enää ole. Uusi sovellus lisätään joko ylhäältä +-pikavalikosta tai projektin **+Add**-sivulta.



Nopein reitti: + (ylhäällä) → *Container images*.

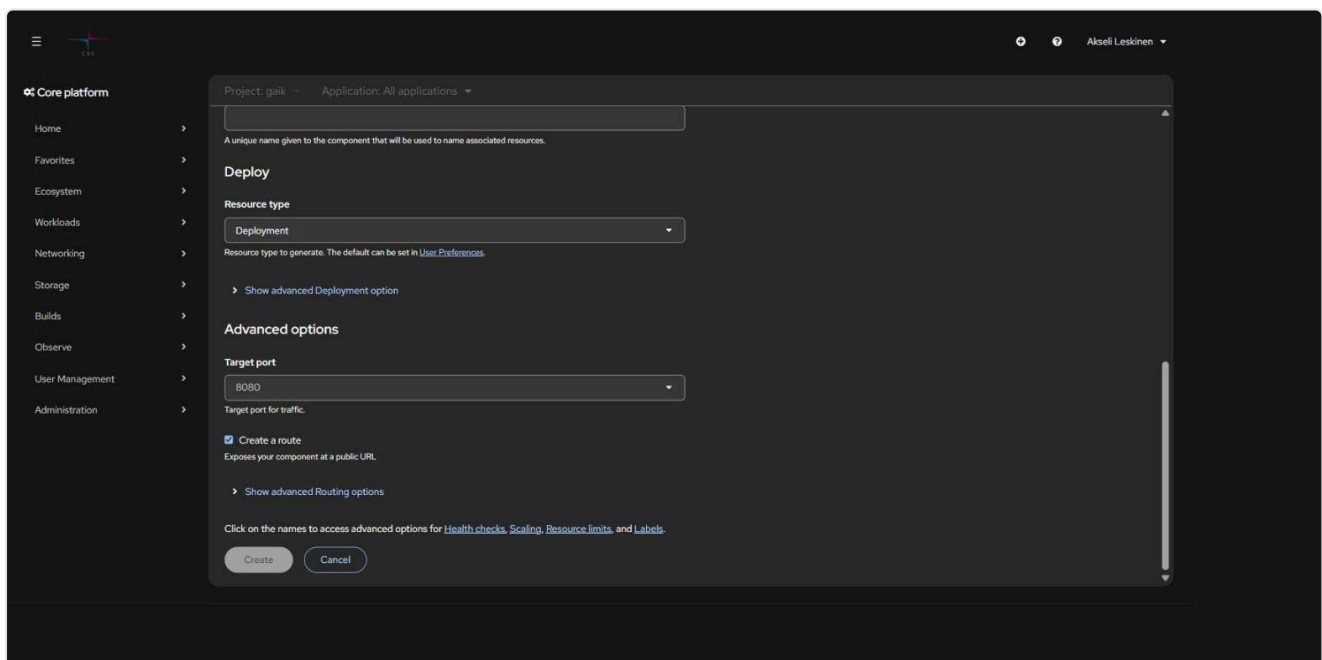


Deploy Image -lomake:



- **Image name from external registry** — kirjoita esim. `käyttäjä/sovellus:latest` (Docker Hub) tai `satama.csc.fi/projekti/sovellus:1.0`
- **Image stream tag from internal registry** — valitse aiemmin työnnetty image
- **Select project** — kohdeprojekti
- **Application** — looginen ryhmä, johon komponentti kuuluu (näky Topology-näkymässä)
- **Name** — komponentin nimi, josta johdetaan Deployment, Service ja Route

Vieritä alas **Advanced options** -osioon:



- **Target port** — portti, jota sovelluksesi kuuntelee (esim. 3000 tai 8080)
- **Create a route** — luo julkisen HTTPS-osoitteen. Jätä pois, jos palvelu saa olla vain sisäinen (esim. backend tai tietokanta).

Paina **Create**. Rahti luo Deploymentin, Servicen ja mahdollisen Routen, generoi osoitteen `<nimi>-<projekti>.2.rahtiapp.fi` ja hoitaa TLS:n (edge-terminointi, HTTP ohjataan HTTPS:ään).

Julkaisun jälkeen sovellus näkyy **Topology**-näkylässä ryhmiteltynä sovelluksittain:



4. Julkaise komentoriviltä (manifestit)

Kun sovellus deployataan toistuvasti, selainlomake alkaa haitata: konfiguraatio ei ole versionhallinnassa. Suositeltu rakenne:

```
<repo>/openshift/  
manifests/          # deklaratiiiviset objektit - aja `oc apply -f` KERRAN  
  00-imagestreams.yaml  
  10-api-deployment.yaml  11-api-service.yaml  
  20-web-deployment.yaml  21-web-service.yaml  22-web-route.yaml  
deploy.sh           # toistuva silmukka: build → push → rollout
```

- **Kerran:** `oc apply -f openshift/manifests/` ja salaisuudet `oc create secret generic ... --from-literal=...`
- **Joka koodimuutoksella:** `./openshift/deploy.sh`

Minimaalinen Deployment + Service + Route:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: sovellukseni
spec:
  replicas: 1
  selector:
    matchLabels:
      app: sovellukseni
  template:
    metadata:
      labels:
        app: sovellukseni
    spec:
      securityContext: {}          # EI runAsUser - SCC antaa UID:n
      containers:
        - name: sovellukseni      # nimen on täsmättävä image-triggerin kanssa
          image: image-registry.openshift-image-
registry.svc:5000/<projekti>/sovellukseni:latest
          imagePullPolicy: Always
          ports:
            - containerPort: 3000
          resources:
            requests: { cpu: 100m, memory: 256Mi }
            limits:   { cpu: 500m, memory: 512Mi }
          readinessProbe:
            httpGet: { path: /health, port: 3000 }
            initialDelaySeconds: 5
```

```
apiVersion: v1
kind: Service
metadata:
  name: sovellukseni
spec:
  selector:
    app: sovellukseni
  ports:
    - name: http
      port: 3000
      targetPort: 3000
```

```
apiVersion: route.openshift.io/v1
```

```
kind: Route
metadata:
  name: sovellukseni
spec:
  host: sovellukseni.2.rahtiapp.fi
  to:
    kind: Service
    name: sovellukseni
  port:
    targetPort: http
  tls:
    termination: edge
    insecureEdgeTerminationPolicy: Redirect
```

Huomaa, että klusterin sisällä imageen viitataan osoitteella `image-registry.openshift-image-registry.svc:5000/...`, ei julkisella `image-registry.apps.2.rahti.csc.fi` -nimellä.

Kaksi asiaa, jotka kannattaa tietää ennen kuin etsii vikaa väärästä paikasta:

- **resources -lohkon voi jättää pois**kin, jolloin LimitRange antaa oletukset (100m/500m CPU, 500Mi/1Gi muisti). Ne eivät näy Deploymentin specissä vaan vasta podissa: `oc get pod <pod> -o jsonpath='{.spec.containers[0].resources}'`.
- **securityContext: {} on tarkoituksellinen**. SCC `restricted-v2` antaa podille satunnaisen UID:n (esim. `1006240000`) ryhmässä 0. Jos kirjoitat `runAsUser` -arvon, podi hylätään.

Automaattinen uudelleenkäynnistys uudesta imagesta

Pelkkä `docker push` ei käynnistä podia uudelleen. Lisää Deploymentille **image trigger**, niin uusi `:latest` otetaan käyttöön automaattisesti:

```
metadata:
  annotations:
    image.openshift.io/triggers: |
      [{"from":{"kind":"ImageStreamTag","name":"sovellukseni:latest"},
        "fieldPath":"spec.template.spec.containers[?(@.name==\"sovellukseni\")].image"}]
```

Trigger kirjoittaa `.image` -kentän uudelleen `@sha256` -digestiksi, joten elävässä objektissa ei näy `:latest`. Se on oikein, ei vika.

Sama komentoriviltä:

```
oc set triggers deployment/sovellukseni \
  --from-image=sovellukseni:latest -c sovellukseni -n <projekti>
```

Jos deployment ei päivity pushin jälkeen, katso [09 Vianmäärittäminen](#).

Toistuva deploy-skripti

Käytännössä toimiva `deploy.sh` tekee neljä asiaa:

```
#!/usr/bin/env bash
set -euo pipefail

NS=minun-projekti
APP=sovellukseni
REG=image-registry.apps.2.rahti.csc.fi
TAG=$(date +%Y%m%d-%H%M%S)

docker build --platform linux/amd64 -t "$APP" .
docker tag "$APP" "$REG/$NS/$APP:latest"
docker tag "$APP" "$REG/$NS/$APP:$TAG"
docker push "$REG/$NS/$APP:latest"
docker push "$REG/$NS/$APP:$TAG"

# Pakota rullaus myös silloin kun digest ei muuttunut (esim. muuttunut Secret)
oc patch deployment/"$APP" -n "$NS" -p \
  '{"spec":{"template":{"metadata":{"annotations":{"kubectl.kubernetes.io/restartedAt":"'$(date -Iseconds)'"}}}}}'
oc rollout status deployment/"$APP" -n "$NS" --timeout=180s
```

Salaisuudet eivät päivity itsestään. Muutettu Secret ei näy ajossa olevalle podille ennen uudelleenkäynnistystä — siksi `restartedAt` -patch.

Binary build vaihtoehtona

Jos et halua ajaa Dockeria paikallisesti, Rahti voi rakentaa imagen puolestasi lähdekoodista:

```
# Kerran
oc new-build --name=sovellukseni --binary --strategy=docker \
  --to=sovellukseni:latest -n <projekti>

# Joka buildissa, projektin juuressa
oc start-build sovellukseni --from-dir=. --follow -n <projekti>
```

Tämä on kätevä testaukseen. Jatkuvaan käyttöön suosittelemme GitHub-integraatiota: [3. GitHub-integraatio](#) →

Edellinen: [1. Aloitus](#) · **Seuraava:** [3. GitHub-integraatio](#) →

3. GitHub-integraatio (BuildConfig ja webhookit)

`git push` → webhook → Rahti rakentaa imagen → sovellus päivittyy. Ei paikallista Dockeria, ei manuaalisia pushia.

Sisälllys

- Milloin tämä kannattaa
- Builder Image vai oma Dockerfile
- Tunnettu virhe: "URL is valid but cannot be reached"
- Vaihe 1: SSH-avain GitHubiin
- Vaihe 2: BuildConfig
- Vaihe 3: Webhook
- Buildien seuranta
- Sudenkuopat

Milloin tämä kannattaa

Tapa	Hyvä	Huono
Docker push paikallisesti (luku 2)	Nopea, täysi kontrolli, toimii offline	Vaatii Dockerin, manuaalinen
BuildConfig + webhook (tämä luku)	Automaattinen, ei paikallista Dockeria, build lokitetaan klusteriin	Build syö projektin kiintiötä, hitaampi, virheet debugataan build-logeista
GitHub Actions → docker push	Tuttu putki, testit samassa	Vaatii Rahti-tokenin CI:n salaisuuksiin

Builder Image vai oma Dockerfile

Builder Image (S2I) — Rahti pakkaa lähdekoodin valmiiseen ajoympäristöön. Ei Dockerfilea, nopea aloittaa, riittää tavalliselle Node.js- tai Python-sovellukselle.

Oma Dockerfile (Docker strategy) — täysi kontrolli, monivaiheiset buildit, välttämätön esim. Vite/React-frontendille. Vaatii alla kuvatun kiertotien.

Builder imagen versiot vanhenevat. Valitse listasta ajantasainen UBI-pohjainen versio; vuosia vanha Node.js 18 -builder ei enää saa tietoturvapäivityksiä.

Tunnettu virhe: "URL is valid but cannot be reached"

Kun projektin yrittää luoda suoraan *Import from Git* -toiminnolla käyttäen SSH-osoitetta ja Dockerfile-strategiaa, Rahti näyttää virheen **"URL is valid but cannot be reached"** vaikka SSH-avain olisi oikein sekä GitHubissa että Rahdissa.

Kiertotie:

1. Luo sovellus ensin **Builder Imagella**. Punainen "URL is valid but cannot be reached" jää näkyviin kentän alle, mutta se on pelkkä varoitus: **Create**-painike toimii ja projekti syntyy normaalisti. Virhe koskee vain Dockerfile-strategian validointia.
2. Muokkaa sen jälkeen BuildConfigia (*Builds* → *BuildConfigs* → *Actions* → *Edit BuildConfig* tai suoraan YAML) Docker-strategiaan:

```
strategy:
  type: Docker
  dockerStrategy:
    dockerfilePath: Dockerfile
```

Vaihtoehtoisesti luo BuildConfig kokonaan komentoriviltä, jolloin lomakkeen validointibugi ei häiritse (ks. vaihe 2).

Vaihe 1: SSH-avain GitHubiin

Julkiselle repositoriolle riittää HTTPS-osoite eikä avainta tarvita. Yksityiselle:

```
# 1. Luo avainpari (tyhjä passphrase – Rahti ei osaa kysyä sitä)
ssh-keygen -t ed25519 -C "rahti-deploy" -f ./rahti_github_key

# 2. Vie julkinen avain GitHubiin:
#   repo → Settings → Deploy keys → Add deploy key → liitä rahti_github_key.pub
#   Jätä "Allow write access" pois päältä – buildi tarvitsee vain lukuoikeuden.

# 3. Vie yksityinen avain Rahtiin salaisuutena
oc create secret generic github-ssh-key \
  --type=kubernetes.io/ssh-auth \
  --from-file=ssh-privatekey=./rahti_github_key -n <projekti>

oc secrets link builder github-ssh-key -n <projekti>

# 4. Poista yksityinen avain levyltä
rm rahti_github_key
```

Selaimessa sama tehdään *Import from Git* -lomakkeen kohdassa **Show advanced Git options** → **Source Secret** → **Create new Secret** (tyyppi *SSH key*). Liitä avain kokonaisuudessaan, myös **BEGIN** - ja **END** -rivit.

Vaihe 2: BuildConfig

```
apiVersion: build.openshift.io/v1
kind: BuildConfig
metadata:
  name: sovellukseni
spec:
  source:
    type: Git
    git:
      uri: git@github.com:<org>/<repo>.git
      ref: main # ks. sudenkuopat!
    sourceSecret:
      name: github-ssh-key
  strategy:
    type: Docker
    dockerStrategy:
      dockerfilePath: Dockerfile
  output:
    to:
      kind: ImageStreamTag
      name: sovellukseni:latest
  triggers:
    - type: ConfigChange
    - type: GitHub
      github:
        secretReference:
          name: github-webhook-secret
---
apiVersion: image.openshift.io/v1
kind: ImageStream
metadata:
  name: sovellukseni
```

```
# Webhook-salaisuus ensin
oc create secret generic github-webhook-secret \
  --from-literal=WebHookSecretKey=$(openssl rand -hex 20) -n <projekti>

oc apply -f buildconfig.yaml -n <projekti>
```

Vaihe 3: Webhook

1. Hae URL Rahdista: *Builds* → *BuildConfigs* → *<nimi>* → *Webhooks* → **Copy URL with Secret**

Komentoriviltä:

```
oc describe bc/sovellukseni -n <projekti> | grep -A2 "Webhook"
```

Muoto:

```
https://api.2.rahti.csc.fi:6443/apis/build.openshift.io/v1/namespaces/<projekti>/buildconfigs/<bc>/webhooks/<secret>/github
```

Yleiskäyttöinen (ei-GitHub) päätepiste päättyy `/generic`. Valitse tyyppi sen mukaan, missä koodi on: GitHub, GitLab, Bitbucket vai Generic.

2. Lisää webhook GitHubiin: *repo* → *Settings* → *Webhooks* → *Add webhook*

- **Payload URL:** kopioitu osoite
- **Content type:** `application/json`
- **Secret:** sama salaisuus kuin URL:ssä
- **SSL verification:** Enable
- **Which events:** *Just the push event*
- **Active:** ✓

3. Testaa: tee muutos, pushaa, ja katso käynnistyykö buildi.

Buildien seuranta

```
oc get builds -n <projekti>          # kaikki buildit
oc logs build/<build-nimi> -n <projekti>  # yhden buildin loki
oc logs -f bc/sovellukseni -n <projekti>  # seuraa uusinta buildia
oc start-build sovellukseni -n <projekti>  # käynnistä käsin
```

Sudenkuopat

Haaran nimi. Rahdin BuildConfigin oletushaara on `master`, GitHubin `main`. Jos `ref`-kenttää ei ole asetettu, `main`-haaran pushit jäävät huomiotta ilman virheilmoitusta. Aseta `spec.source.git.ref: main` — tai selaimessa *Show advanced Git options* → *Git reference*.

Epäonnistunut buildi on hiljainen vika. Kaatunut buildi ei riko ajossa olevaa sovellusta: uutta imagea ei vain synny ja podi ajaa vanhaa koodia. Podi näyttää `Ready 1/1` ja route palauttaa 200 — mutta koodimuutos ei näy. Tarkista aina `oc get builds` kun "deploy ei mennyt läpi".

Build syö kiintiötä. Build-podi varaa CPU:n ja muistin samasta laskentaprojektin kiintiöstä kuin sovellukset. Jos kiintiö on täynnä, build jää `Pending`-tilaan.

Build-podit kasautuvat. Jokainen ajo jättää `<sovellus>-<n>-build` -podin. `Completed / Succeeded` on normaali, `Failed` on aito löydös. Siivoa vanhat tarvittaessa:

```
oc delete pods -n <projekti> --field-selector=status.phase==Succeeded
```

Yksityisen imagen pull. Jos BuildConfig kirjoittaa Satamaan tai vetää pohjaimagen yksityisestä rekisteristä, linkitä pull secret myös `builder`-tunnukselle:

```
oc secrets link builder <pull-secret> -n <projekti>
```

Edellinen: [2. Julkaisu](#) · **Seuraava:** [4. Reitit ja verkko](#) →

Lähteet: [CSC: Webhooks](#) · [CSC: Deploy from Git](#)

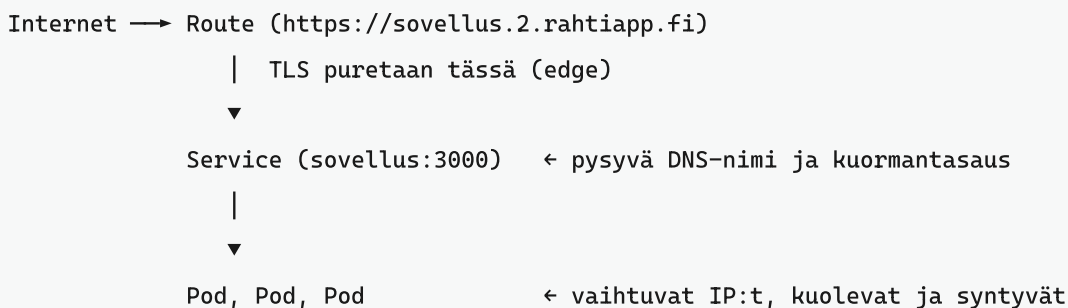
4. Reitit, verkko ja URL-osoitteet

Miten sovellus saa julkisen osoitteen, miten palvelut puhuvat keskenään klusterin sisällä, ja mitä tehdä kun pitkä pyyntö katkeaa 30 sekunnissa.

Sisällys

- Kolme tasoa: Pod, Service, Route
- Sisäinen liikenne: Service-DNS
- Julkinen osoite: Route
- Oman URL-osoitteen luonti
- Reitin vaihto ilman katkosta
- TLS-terminointi
- 504 Gateway Time-out pitkissä pyynnöissä
- IP-rajaus ja muut annotaatiot
- Oma verkkotunnus
- Ulosmenevä liikenne

Kolme tasoa: Pod, Service, Route



- **Podin IP vaihtuu** aina kun podi käynnistyy uudelleen. Älä käytä sitä mihinkään.
- **Service** on pysyvä nimi ja IP, joka reitittää liikenteen label-valitsimen perusteella.
- **Route** on ainoa asia, joka näkyy internetiin. Ilman Routea palvelu on tavoitettavissa vain klusterin sisältä — mikä on backendille ja tietokannalle **oikea** valinta.

Oletuksena podi voi puhua vain oman projektinsa podien kanssa (NetworkPolicy).

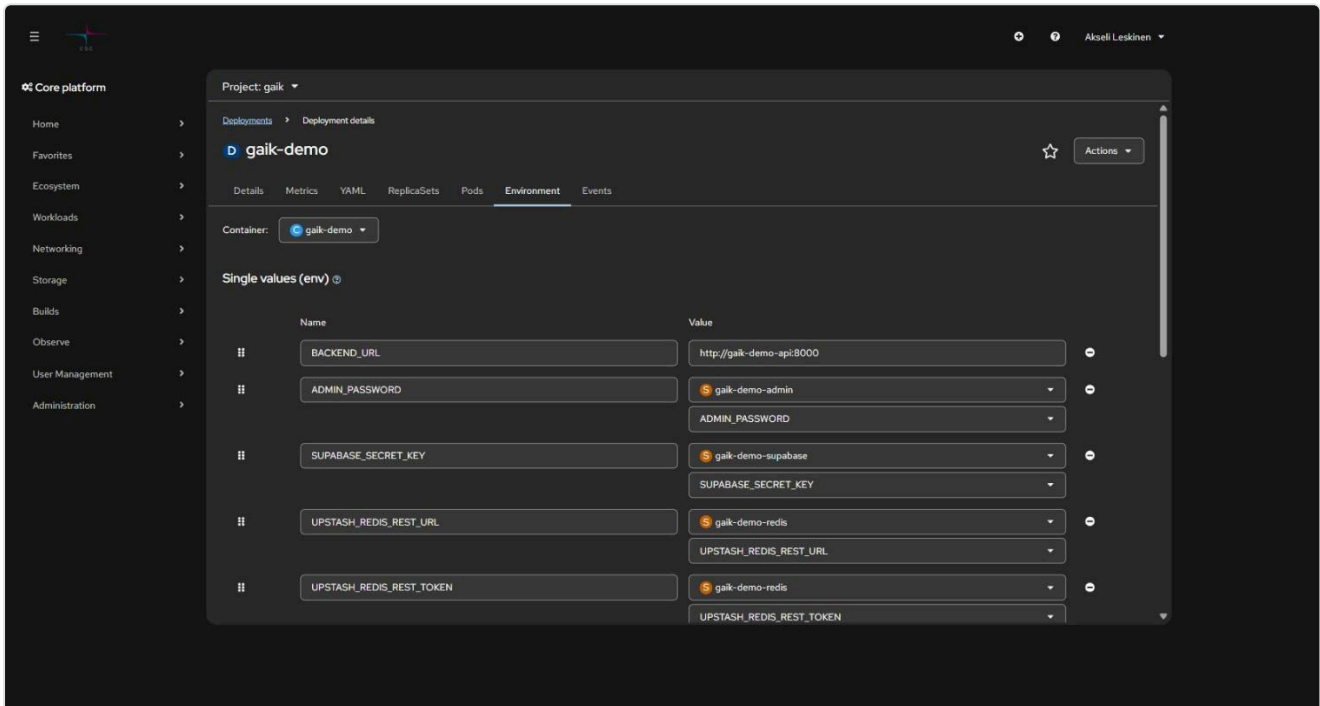
Sisäinen liikenne: Service-DNS

Jokainen Service saa DNS-nimen:

```
<service>.<projekti>.svc.cluster.local      # täysi nimi
<service>.<projekti>                        # toisesta projektista
<service>                                    # saman projektin sisältä
```

Käytännössä frontend kutsuu backendiä näin:

```
oc set env deployment/frontend -n <projekti> \
  BACKEND_URL=http://backend-api:8000
```



Huomaa `http://`, ei `https://` — TLS puretaan jo Routella, ja sisäverkko on klusterin sisäistä liikennettä. Portti on **Services** portti, ei Routen.

Yhteyden testaus podista:

```
oc exec -it deployment/frontend -n <projekti> -- sh
# podin sisällä:
wget -qO- http://backend-api:8000/health
```

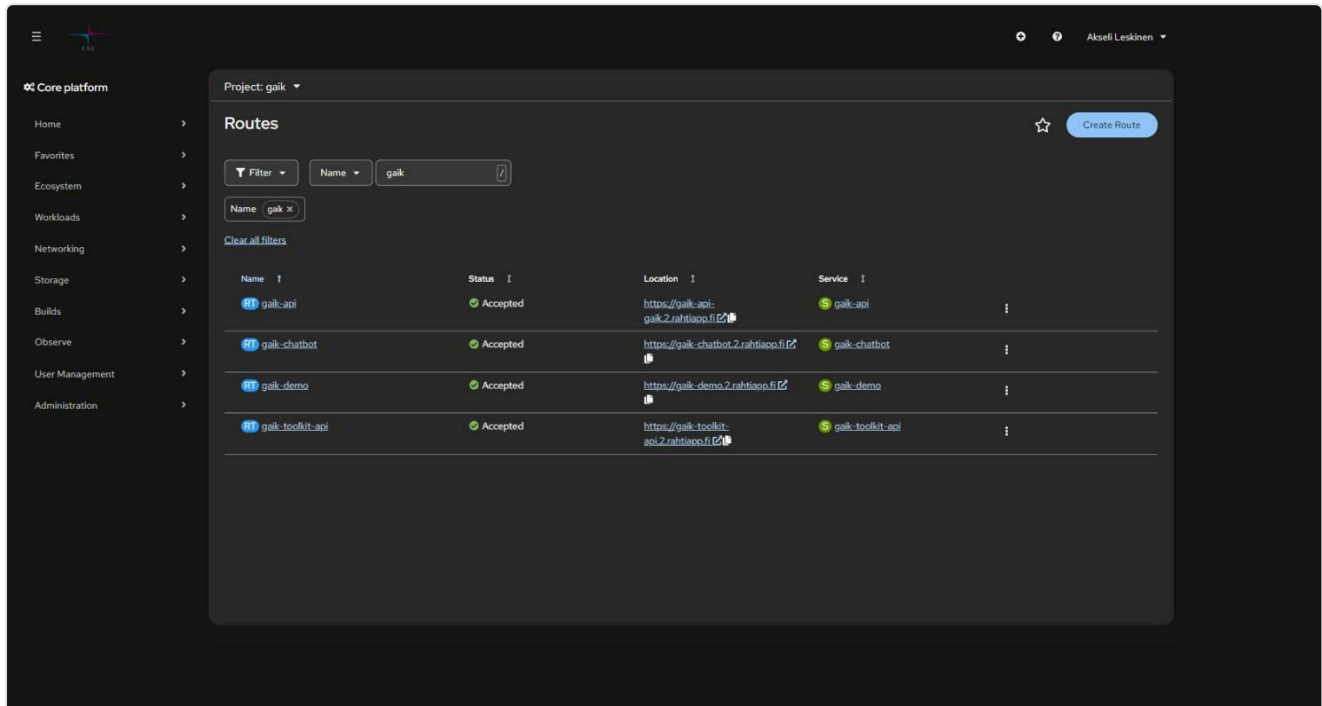
Paras käytäntö: älä anna backendille Routea lainkaan. Silloin sitä ei voi kutsua internetistä eikä autentikointia tarvitse rakentaa kahteen kertaan.

Lisää kuvioita: [06 Frontend ja backend](#).

Julkinen osoite: Route

Kun luot sovelluksen konsolista ja rastitat *Create a route*, Rahti generoi osoitteen muodossa:

```
https://<komponentin-nimi>-<projekti>.2.rahtiapp.fi
```



Kuvassa `gaik-api` käyttää generoitua nimeä (`gaik-api-gaik.2.rahtiapp.fi`) ja muut räätälöityä (`gaik-demo.2.rahtiapp.fi`). Molemmat toimivat rinnakkain.

```
oc get routes -n <projekti>
oc get route <nimi> -n <projekti> -o jsonpath='{.spec.host}'
```

Oman URL-osoitteen luonti

Osoitteen etuliite on **uniikki koko Rahti-ympäristössä** ja päätte on oltava `.2.rahtiapp.fi`. Reitti on oma objektinsa, joten uuden osoitteen saa luomalla uuden Routen — vanhaa ei tarvitse poistaa.

```
# uusi-reitti.yaml
apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: <reitin-nimi>
  namespace: <projekti>
  labels:
    app: <sovelluksen-nimi>
  annotations:
    openshift.io/host.generated: "false"
spec:
  host: <haluttu-nimi>.2.rahtiapp.fi
  to:
    kind: Service
    name: <palvelun-nimi>
    weight: 100
  port:
    targetPort: <portin-nimi-tai-numero>
  tls:
    termination: edge
    insecureEdgeTerminationPolicy: Redirect
  wildcardPolicy: None
```

```
oc project                # varmista projekti
oc apply -f uusi-reitti.yaml
curl -I https://<haluttu-nimi>.2.rahtiapp.fi
```

Esimerkki. Sovellus `oppimisavustaja`, jonka Service on `oppimisavustaja` portissa 3000:

```
spec:
  host: oppimisavustaja.2.rahtiapp.fi
  to:
    kind: Service
    name: oppimisavustaja
  port:
    targetPort: 3000-tcp
```

`targetPort` viittaa **Service**n porttiin. Konsolista luoduilla palveluilla portin nimi on tyypillisesti `<numero>-tcp`. Tarkista: `oc get svc <nimi> -o yaml`.

Sama yhdellä komennolla ilman YAML-tiedostoa:

```
oc create route edge <reitini-nimi> \
  --service=<palvelu> --hostname=<nimi>.2.rahtiapp.fi \
  --insecure-policy=Redirect -n <projekti>
```

Reitin vaihto ilman katkosta

Useampi Route voi osoittaa samaan Serviceen. Siirtymä tehdään näin:

1. Luo uusi Route uudella hostnamella
2. Testaa se
3. Päivitä linkit ja mahdolliset CORS-asetukset
4. Poista vanha vasta kun mikään ei enää käytä sitä

```
oc delete route <vanha-reitti> -n <projekti>
```

TLS-terminointi

Tapa	Mitä tapahtuu	Milloin
edge	Router purkaa TLS:n ja välittää podille HTTP:tä	Oletus. Sovelluksen ei tarvitse tuntea sertifikaatteja.
passthrough	Router ei pura mitään; podi hoitaa TLS:n	Päästä päähän -salaus, oma sertifikaatti podissa
reencrypt	Router purkaa ja salaa uudelleen podille	Sisäverkkokin salattava, mutta ulospäin Rahdin sertifikaatti

`*.2.rahtiapp.fi` -osoitteille Rahdin wildcard-sertifikaatti riittää — omaa sertifikaattia ei tarvitse hankkia. **Route ilman `tls` -lohkoa palvelee pelkkänä HTTP:nä**, joten edge on pyydettävä erikseen.

504 Gateway Time-out pitkissä pyynnöissä

Routen HAProxy-timeout on oletuksena **30 sekuntia**. Pitkä synkroninen pyyntö — LLM-kutsu, iso tiedostolataus, raskas raportti — katkeaa 504-virheeseen, jonka sovellus usein näyttää geneerisenä "Request failed" -viestinä.

```
oc annotate route <reitti> \
  haproxy.router.openshift.io/timeout=120s -n <projekti> --overwrite
```

Tai manifestissa:

```
metadata:
  annotations:
    haproxy.router.openshift.io/timeout: 120s
```

Varmista ensin, että pyyntö oikeasti kestää niin kauan — **504 tasan ~30 sekunnissa** on tämän vian tunnusmerkki:

```
curl -o /dev/null -s -w 'HTTP %{http_code} in %{time_total}s\n' https://<osoite>/hidas
```

Todella pitkille töille parempi ratkaisu on asynkroninen job + tilakysely kuin yhteyden pitäminen auki minuuttikausia.

IP-rajaus ja muut annotaatiot

```
# Salli vain korkeakouluverkko
oc annotate route <reitti> \
  haproxy.router.openshift.io/ip_allowlist='193.166.0.0/16' -n <projekti>

# HSTS (pakota HTTPS selaimessa)
oc annotate route <reitti> \
  haproxy.router.openshift.io/hsts_header="max-age=31536000;includeSubDomains;preload" -n
<projekti>
```

Varoitus: virheellinen allowlist-arvo hylätään hiljaisesti ja **kaikki liikenne sallitaan**. Arvot erotellaan välilyönnillä, ei pilkulla, eikä perässä saa olla ylimääräistä välilyöntiä.

Oma verkkotunnus

Oman domainin saa käyttöön osoittamalla DNS:n Rahdin routeriin:

```
CNAME <oma-domain> → router-default.apps.2.rahti.csc.fi
```

Jos CNAME ei ole mahdollinen (juuridomain), käytä A-tietuetta kyseisen nimen IP:llä — mutta muista, että IP voi muuttua. Sertifikaatti on omalla vastuulla; Let's Encrypt -automaatio onnistuu Rahdin cert-manager-tuella. Ohjeet: [CSC: Custom domains](#).

Ulosmenevä liikenne

Kaikki Rahdista ulos lähtevä liikenne käyttää tällä hetkellä IP-osoitetta `86.50.229.150` . Tarvitset sitä, jos ulkoinen tietokanta tai API on palomuurin takana.

CSC varoittaa, että osoite voi muuttua, ja rinnakkain ajettavilla Rahti-versioilla on eri osoite. Älä kovakoodaa sitä ilman suunnitelmaa päivittämisestä — tarkista arvo aina [dokumentaatiosta](#). Oman, projektikohtaisen egress-IP:n voi pyytää palvelupisteestä.

5. Ympäristömuuttujat ja salaisuudet

Yleisin yksittäinen virhe Rahti-projekteissa on sekoittaa build-aikainen ja ajonaikainen muuttuja. Se johtaa joko rikkinäiseen buildiin tai selaimeen vuotaneeseen API-avaimeen.

Sisällys

- Build-aika vs. ajonaika
- Ajonaikaisten arvojen asettaminen
- Salaisuudet (Secret)
- ConfigMap ei-salaiselle konfiguraatiolle
- Sudenkuopat
- Muuttujien tarkastelu konsolissa

Build-aika vs. ajonaika

Muuttujan tyyppi	Milloin asetetaan	Missä se päättyy
NEXT_PUBLIC_*, VITE_*, REACT_APP_*	docker build	Paistetaan JS-bundleen — näkyy jokaiselle sivun kävijälle
Kaikki muut	Podin käynnistyessä (oc set env , Secret)	Vain palvelinpuolella

Kaksi seurausta:

1. **Salaisuus** NEXT_PUBLIC_-etuliitteellä on julkinen. Se on selaimen lähdekoodissa. Jos API-avain on päätenyt sinne, se on kierrätettävä.
2. **Ajonaikainen muuttuja, jota build tarvitsee, kaataa buildin.** Esimerkiksi Next.js:n next build ajaa sivut läpi ja kaatuu, jos konfiguraatio vaatii avaimen, jota ei ole. Ratkaisu on antaa buildille tyhjä paikanpitäjä ja oikea arvo vasta ajossa:

```
# Build hyväksyy tyhjän arvon, oikea tulee Secretistä podin käynnistyessä
ARG DATABASE_URL=""
ENV DATABASE_URL=$DATABASE_URL
RUN npm run build
```

Ajonaikaisten arvojen asettaminen

```
# Yksittäisiä arvoja
oc set env deployment/sovellus -n <projekti> \
  NODE_ENV=production BACKEND_URL=http://backend-api:8000

# Kaikki Secretin avaimet kerralla
oc set env deployment/sovellus --from=secret/sovellus-env -n <projekti>

# ConfigMapista
oc set env deployment/sovellus --from=configmap/sovellus-config -n <projekti>

# Listaa nykyiset
oc set env deployment/sovellus -n <projekti> --list

# Poista (huomaa loppuviiva)
oc set env deployment/sovellus -n <projekti> VANHA_MUUTTUJA-
```

Jokainen `oc set env` käynnistää rullaavan päivityksen automaattisesti.

Salaisuudet (Secret)

Salasanat, API-avaimet ja tokenit kuuluvat Secretiin — eivät Deploymentin `env` -lohkoon eivätkä versionhallintaan.

```
# Suositeltu: yksi avain kerrallaan, ei jäsennysyllätyksiä
oc create secret generic sovellus-env \
  --from-literal=DATABASE_URL='postgresql://user:salasana@postgres:5432/db' \
  --from-literal=OPENAI_API_KEY='sk-...' \
  -n <projekti>

# Tai gitignoratusta tiedostosta (lue sudenkuopat ensin!)
oc create secret generic sovellus-env --from-env-file=.env.local -n <projekti>

# Kytke Deploymenttiin
oc set env deployment/sovellus --from=secret/sovellus-env -n <projekti>
```

Yksittäinen avain vain tiettyyn muuttujaan:

```
env:
  - name: DATABASE_URL
    valueFrom:
      secretKeyRef:
        name: sovellus-env
        key: DATABASE_URL
```

Arvon tarkistus (kannattaa aina tehdä luonnin jälkeen):

```
oc get secret sovellus-env -n <projekti> -o jsonpath='{.data.DATABASE_URL}' | base64 -d
```

Päivitys ja kierrätys:

```
oc create secret generic sovellus-env \
  --from-literal=OPENAI_API_KEY='sk-uusi' \
  --dry-run=client -o yaml | oc apply -f -

oc rollout restart deployment/sovellus -n <projekti>
```

ConfigMap ei-salaiselle konfiguraatiolle

```
oc create configmap sovellus-config \
  --from-literal=LOG_LEVEL=info \
  --from-literal=FEATURE_X=true -n <projekti>

oc set env deployment/sovellus --from=configmap/sovellus-config -n <projekti>
```

ConfigMapin sisältö näkyy kaikille, joilla on lukuoikeus projektiin — se ei ole salaisuus, vain rakenteinen konfiguraatio.

Sudenkuopat

`--from-env-file` säilyttää rivin lopun kommentit. Rivi

```
MODEL=gpt-5.6-sol # nopein
```

tallentaa arvoksi `gpt-5.6-sol # nopein`, ja sovellus hajoaa hämärästi. Käytä `--from-literal` -muotoa tai siivoa tiedosto ensin. Tarkista aina `base64 -d` :llä.

Salaisuudet eivät päivity lennossa. Muutettu Secret ei näy ajossa olevalle podille. Vasta `oc rollout restart` tuo uudet arvot.

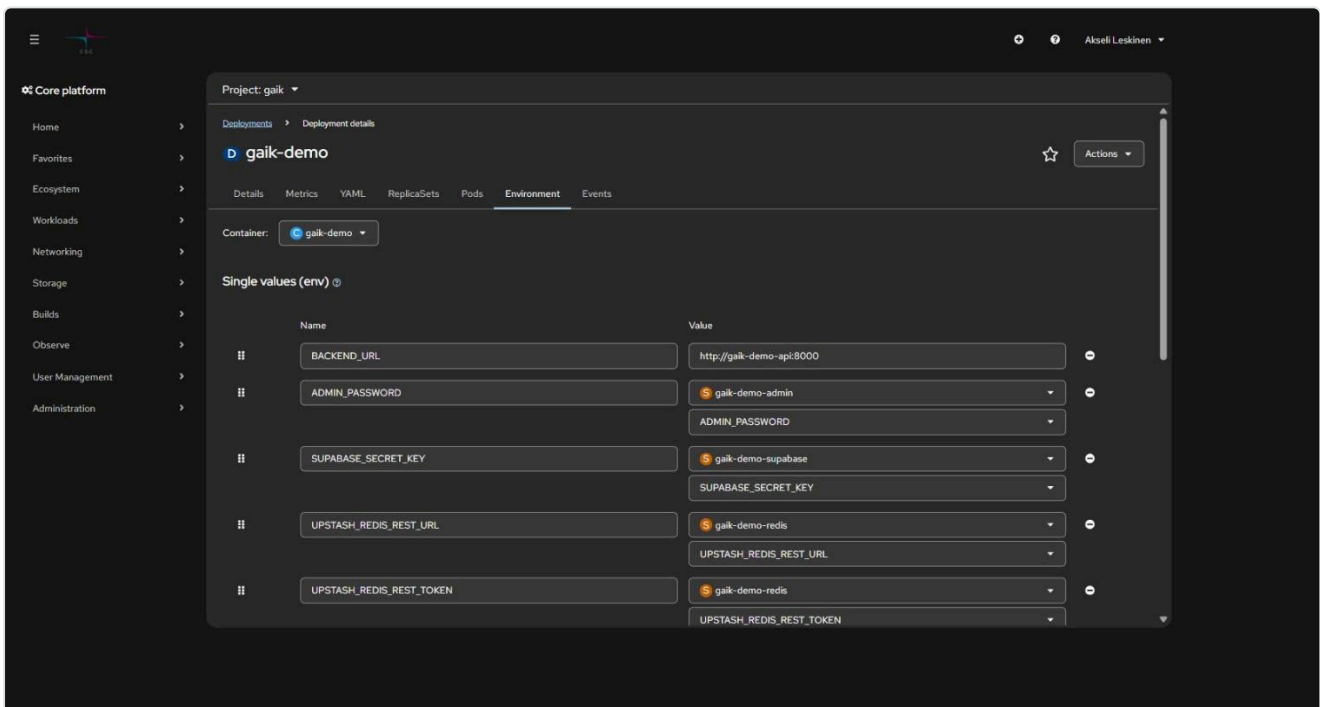
Lainausmerkit. Erikoismerkkejä sisältävä salasana kannattaa laittaa yksinkertaisiin lainausmerkkeihin, jotta shell ei tulkitse niitä: `--from-literal=PW='p@$w0rd!'`.

.env versionhallintaan. Varmista, että `.gitignore` sisältää `.env` ja `.env.local`. Jos salaisuus on jo commitoitu, sen poistaminen historiasta ei riitä — avain on kierrätettävä.

Kubernetes-Secret ei ole salattu, vaan base64-koodattu. Kuka tahansa, jolla on lukuoikeus projektiin, näkee arvon. Rajoita projektin oikeudet vain niille, jotka niitä tarvitsevat (*User Management* → *RoleBindings*).

Muuttujien tarkastelu konsolissa

Workloads → *Deployments* → *<sovellus>* → *Environment*:



Kuvassa `BACKEND_URL` on tavallinen arvo ja loput haetaan Secretistä — Secretin nimi ja avain näkyvät, arvo ei. Tämä on juuri se tapa, jolla salaisuudet halutaan näkyvän.

6. Frontend ja backend Rahdissa

Kolme tapaa julkaista React (Vite) + Node.js -sovellus, ja miten palvelut kannattaa kytkeä toisiinsa.

Sisälllys

- Valitse arkkitehtuuri
- Vaihtoehto A: yksi palvelu, backend tarjoilee frontendin
- Vaihtoehto B: monorepo yhdessä kontissa
- Vaihtoehto C: erilliset palvelut
- API-polut eri malleissa
- CORS
- Frameworkilla ei ole väliä
- Terveystarkistukset ja Basic Auth
- Portit

Valitse arkkitehtuuri

	A: Yksi palvelu	B: Monorepo, yksi kontti	C: Erilliset palvelut
Kontteja	1	1	2+
CORS tarvitaan	ei	ei	kyllä (tai sisäinen kutsu)
Skaalautuu erikseen	ei	ei	kyllä
Deploy-monimutkaisuus	pieni	pieni	keskitaso
Sopii	pieni sovellus, demo	tiimin monorepo	tuotanto, eri teknologiat

Jos et osaa päättää: aloita **A**:sta tai **B**:stä ja siirry **C**:hen kun frontend ja backend alkavat elää eri tahtia.

Vaihtoehto A: yksi palvelu, backend tarjoilee frontendin

Buildaa frontend staattiseksi ja anna backendin tarjoilla se. Yksi portti, yksi osoite, ei CORSia.

```
cd frontend && npm run build          # tuottaa dist/  
cp -r dist ../backend/client/dist    # backendin staattinen kansio
```

```
// backend/index.js
const express = require("express");
const path = require("path");
const app = require("./app");

// Staattinen frontend
app.use(express.static(path.join(__dirname, "client/dist")));

// TÄMÄN on oltava API-reittien JÄLKEEN, muuten se nappaa myös /api-kutsut.
// app.use toimii sekä Expressissä 4 että 5 (ks. huomio alla).
app.use((req, res) => {
  res.sendFile(path.join(__dirname, "client/dist", "index.html"));
});

const PORT = process.env.PORT || 8080;
app.listen(PORT, () => console.log(`Server running on ${PORT}`));
```

Express 5 rikkoi `app.get("*")`. Netin ohjeissa SPA:n fallback-reitti kirjoitetaan lähes aina muotoon `app.get("*", ...)`. Se toimii Expressissä 4, mutta **kaatuu Expressissä 5** virheeseen `TypeError: Missing parameter name`. Express 5 on npm:n oletusversio, eli tuore `npm install express` saa sen. Yllä oleva `app.use` toimii molemmissa; jos haluat nimenomaan reitin, Express 5:n muoto on `app.get("/splat", ...)`.

Monorepossa kopiointi kannattaa automatisoida juuren `package.json`-tiedostoon:

```
{
  "scripts": {
    "build:frontend": "cd frontend && npm install && npm run build",
    "postbuild:frontend": "rm -rf backend/client/dist && cp -r frontend/dist backend/client/",
    "build": "npm run build:frontend",
    "start": "cd backend && npm start"
  }
}
```

Vaihtoehto B: monorepo yhdessä kontissa

Sama lopputulos kuin A:ssa, mutta kopiointi tapahtuu Dockerfilessa, joten paikallista build-vaihetta ei tarvita.

```
project/
├─ backend/      (index.js, package.json)
├─ frontend/     (src/, package.json)
└─ Dockerfile
```

```
# --- Build ---
FROM node:24-alpine AS builder
WORKDIR /app
COPY package*.json ./
COPY frontend/package*.json ./frontend/
COPY backend/package*.json ./backend/
RUN cd frontend && npm ci && cd ../backend && npm ci
COPY . .
RUN cd frontend && npm run build
RUN mkdir -p backend/client && cp -r frontend/dist backend/client/

# --- Runtime ---
FROM node:24-alpine
WORKDIR /app
COPY --from=builder /app/backend ./backend
RUN chown -R 1001:0 /app && chmod -R g+rwX /app
USER 1001
EXPOSE 8080
CMD ["node", "backend/index.js"]
```

`docker-compose.yml` on kätevä paikalliseen kehitykseen, mutta **Rahti ei aja compose-tiedostoja**. Se lukee Dockerfilen ja Kubernetes-manifestit.

Vaihtoehto C: erilliset palvelut

Frontend ja backend ovat omia Deploymenttejaan. Tärkein päätös: **anna backendille Route vai ei**.

```
Internet → Route → frontend-Service → frontend-Pod
                                   | http://backend-api:8000
                                   ▼
                               backend-Service → backend-Pod
                               (ei Routea = ei näy internetiin)
```

Sisäinen kutsu palvelunimellä:

```
oc set env deployment/frontend -n <projekti> \
  BACKEND_URL=http://backend-api:8000
```

```
// Palvelinpuolen koodissa (Next.js route handler, Express-proxy, ...)  
const backendUrl = process.env.BACKEND_URL || "http://localhost:8000";  
const res = await fetch(`${backendUrl}/api/data`);
```

Frontendin Dockerfile (Vite, staattinen tarjoilu):

```
FROM node:24-alpine AS builder  
WORKDIR /app  
COPY package*.json ./  
RUN npm ci  
COPY . .  
RUN npm run build  
  
FROM node:24-alpine  
WORKDIR /app  
COPY --from=builder /app/dist ./dist  
RUN npm install -g serve && chown -R 1001:0 /app && chmod -R g+rwX /app  
USER 1001  
EXPOSE 8080  
CMD ["serve", "-s", "dist", "-p", "8080"]
```

`serve -s` (single) ohjaa kaikki polut `index.html`:ään, mikä on välttämätöntä client-side-reitityksessä (React Router). **Jos sovellus ei ole SPA**, jätä `-s` pois: `CMD ["serve", "dist", "-p", "8080"]`.

Muista `.dockerignore`:

```
node_modules  
dist  
.env
```

Selaimesta tehtävä kutsu ei voi käyttää sisäistä nimeä. `http://backend-api:8000` toimii vain podin sisältä. Jos frontend on täysin staattinen ja kutsuu API:a selaimesta, backend tarvitsee joko oman Routen (ja CORSin) tai frontendin palvelinpuolen proxyyn.

API-polut eri malleissa

A ja B — sama origin, käytä suhteellisia polkuja:

```
const res = await fetch("/api/endpoint", { method: "POST", body: formData });
```

C — täysi URL konfiguraatiosta:

```
// src/config.js
export const BACKEND_URL = import.meta.env.VITE_BACKEND_URL || "http://localhost:8080";
```

Muista: `VITE_*` paistetaan bundleen build-aikana (ks. [05 Ympäristömuuttujat](#)) — sinne ei laiteta salaisuuksia.

CORS

CORS koskee vain **selaimen** tekemiä kutsuja eri originiin. Palvelinpuolelta tehty `fetch` (Next.js route handler, Express-proxy, mikä tahansa podista lähtevä kutsu) ei välitä CORSista lainkaan.

Sama päädomain ei tarkoita samaa originia. `https://frontend.2.rahtiapp.fi` ja `https://backend.2.rahtiapp.fi` ovat eri origineja, koska origin on protokolla + koko hostname + portti. Jaettu `.2.rahtiapp.fi` -pääte ei auta.

Paras ratkaisu on välttää CORS kokonaan: älä anna backendille Routea, vaan kutsu sitä frontendin palvelimelta sisäisellä nimellä (`http://backend-api:8000`). Silloin selain näkee vain yhden originin, CORSia ei tarvita ja backend ei ole internetissä.

Jos backend kuitenkin tarvitsee julkisen Routen:

```
app.use(
  cors({
    origin: ["https://frontend.2.rahtiapp.fi"],
    credentials: true,
  })
);
```

Kolme asiaa jotka kaatavat CORSin Rahdissa

1. **origin: "*" ja credentials: true eivät toimi yhdessä.** Tämä ei ole tyyliseikka: selain hylkää vastauksen, jossa on `Access-Control-Allow-Origin: *`, jos pyyntö tehtiin `credentials: "include"` -tilassa. Listaa originit eksplisiittisesti.
2. **Preflight-pyyntö unohtuu.** Kutsu, jossa on `Authorization` -header tai `Content-Type: application/json`, saa selaimen lähettämään ensin `OPTIONS` -pyynnön. Jos backend ei vastaa siihen 2xx-koodilla ja oikeilla headereilla, varsinaista pyyntöä ei koskaan lähetetä. Testaa erikseen:

```
curl -i -X OPTIONS https://backend.2.rahtiapp.fi/api/data -H "Origin: https://frontend.2.rahtiapp.fi" -H "Access-Control-Request-Method: POST" -H "Access-Control-Request-Headers: content-type"
```

Odotettu vastaus on 204 tai 200 ja mukana `Access-Control-Allow-Origin`.

3. **Autentikointi tappaa preflightin.** Selain **ei** lähetä evästeitä eikä `Authorization` -headeria preflight-pyynnössä. Jos sovellus vaatii kirjautumisen kaikilla poluilla (Basic Auth, middleware), `OPTIONS` saa 401:n ja koko kutsu epäonnistuu. Päästä `OPTIONS` läpi autentikoinnista. Sama ansa kuin terveystarkistuksissa alla.

Selainkonsolin virhe *"No 'Access-Control-Allow-Origin' header is present"* tarkoittaa useimmiten yhtä näistä kolmesta, ei sitä että `cors()` -kutsu puuttuisi kokonaan.

Terveystarkistukset ja Basic Auth

Rahti odottaa, että podi kertoo olevansa valmis. Kaksi tapaa:

```
# Avoin /health-polku – paras vaihtoehto
readinessProbe:
  httpGet: { path: /health, port: 8000 }

# Sovellus on kokonaan autentikoinnin takana – HTTP-tarkistus saisi 401
readinessProbe:
  tcpSocket: { port: 3000 }
```

Tämä on aito sudenkuoppa. Jos sovellus (esim. Next.js middleware tai `proxy.ts`) vaatii Basic Authin **kaikilla** poluilla, `httpGet` -tarkistus saa 401, podi ei koskaan siirry Ready-tilaan ja rullaus jää roikkumaan ikuisesti. Käytä joko `tcpSocket` -tarkistusta tai jätä `/health` autentikoinnin ulkopuolelle.

Frameworkilla ei ole väliä

Esimerkeissä on Express ja Vite, koska ne ovat tutuimpia, mutta Rahti ei tiedä mitään frameworkeista. Hono, Fastify, NestJS, FastAPI, Flask, Spring Boot, Go, Rust ja mikä tahansa muu toimii, kunhan kontti täyttää nämä neljä ehtoa:

1. kuuntelee `0.0.0.0`, ei `127.0.0.1`
2. lukee portin `PORT` -ympäristömuuttujasta tai käyttää kiinteää porttia yli 1024
3. ei vaadi rootia eikä tiettyä UID:tä
4. tarjoaa jonkin polun jonka terveystarkistus voi hakea ilman kirjautumista

Esimerkiksi Hono Nodella:

```
import { serve } from "@hono/node-server";
import { Hono } from "hono";

const app = new Hono();
app.get("/health", (c) => c.json({ status: "ok" }));

serve({ fetch: app.fetch, port: Number(process.env.PORT) || 8080, hostname: "0.0.0.0" });
```

Loput tästä ohjeesta (Dockerfile, Service, Route, salaisuudet) on täsmälleen sama riippumatta siitä mitä kirjoitit sisälle.

Portit

- Sovelluksen on kuunneltava `0.0.0.0`, ei `127.0.0.1` — muuten liikenne ei tule perille podin ulkopuolelta.
- Portin numerolla ei sinänsä ole väliä (3000, 8000, 8080), kunhan sama arvo on Dockerfilen `EXPOSE` ssa, Deploymentin `containerPort` issa, Servicen `targetPort` issa ja Routen `targetPort` issa.
- Älä käytä porttia alle 1024. Satunnainen UID ei saa sitoa etuoikeutettua porttia ilman `NET_BIND_SERVICE` -capabilityä, joka on ainoa jonka Rahdissa saa lisätä takaisin. Helpompi tapa on kuunnella 8080:aa.
- Lue portti ympäristömuuttujasta, jos mahdollista: `const PORT = process.env.PORT || 8080`.

Edellinen: [5. Ympäristömuuttujat](#) · **Seuraava:** [7. Tietokanta ja pgvector](#) →

7. PostgreSQL ja pgvector Rahdissa

Tietokanta samaan projektiin sovelluksen kanssa: asennus, pysyvä levy, pgvector ja paikallinen hallinta pgAdminilla.

Sisällys

- Ennen kuin asennat
- Asennus konsolista
- Pysyvä tallennustila (PVC)
- Asennus manifesteilla
- pgvector-laajennuksen käyttöönotto
- Yhteys sovelluksesta
- Paikallinen hallinta port-forwardilla
- Varmuuskopiot

Ennen kuin asennat

Itse ylläpidetty tietokanta Rahdissa käyttää **yhteisöimageja, joille CSC ei anna tukea**. Se sopii demoihin, opetukseen ja kevyeen tuotantoon, mutta kriittiselle datalle kannattaa harkita hallinnoitua tietokantapalvelua.

Kaksi asiaa on pakko tehdä oikein:

1. **Pysyvä levy (PVC) ennen kuin kirjoitat oikeaa dataa.** Ilman sitä koko tietokanta katoaa jokaisella podin uudelleenkäynnistyksellä.
2. **Ei Routea tietokannalle.** Postgres kuuluu sisäverkkoon. Route tekisi siitä internetiin avoimen.

Asennus konsolista

1. **+Add → Container images**
2. **Image name from external registry:**

```
quay.io/rh-aishervices-bu/postgresql-15-pgvector-c9s
```

Tämä on Red Hat AI Services BU:n yhteisöbuildi, jossa pgvector on valmiina ja joka toimii OpenShiftin satunnaisella UID:llä. Tarkista uudempi tagi ennen kuin kiinnität version.

3. **Name:** esim. `postgresql-pgvector`

4. **Ympäristömuuttujat** (Deploymentin luonnin jälkeen *Environment*-välilehdellä, tai suoraan lomakkeen kautta):

Muuttuja	Arvo
POSTGRESQL_USER	postgres
POSTGRESQL_PASSWORD	vahva salasana
POSTGRESQL_DATABASE	vectordb

5. **Poista rasti kohdasta "Create a route".**

Salasana kuuluu Secretiin, ei suoraan lomakkeelle:

```
oc create secret generic postgres-secret \  
  --from-literal=POSTGRESQL_USER=postgres \  
  --from-literal=POSTGRESQL_PASSWORD='<vahva-salasana>' \  
  --from-literal=POSTGRESQL_DATABASE=vectordb -n <projekti>  
  
oc set env deployment/postgresql-pgvector --from=secret/postgres-secret -n <projekti>
```

Pysyvä tallennustila (PVC)

Selaimessa: avaa Deployment → *Actions* → *Add Storage*

- Nimi: `pgvector-data`
- Access mode: **Single user (RWO)**
- Koko: 5 GiB tai enemmän (kiintiö sallii oletuksena yhteensä 100 GiB)
- Mount path: `/var/lib/pgsql/data`

Komentoriviltä:

```
oc set volume deployment/postgresql-pgvector \  
  --add --name=pgvector-data \  
  --type=pvc --claim-name=pgvector-data \  
  --claim-size=5Gi --claim-mode=ReadWriteOnce \  
  --mount-path=/var/lib/pgsql/data -n <projekti>
```

Tarkista:

```
oc get pvc -n <projekti>
```

Asennus manifesteilla

Toistettavaa asennusta varten sama deklaratiiivisesti:

```

apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pgvector-data
spec:
  accessModes: [ReadWriteOnce]
  resources:
    requests:
      storage: 5Gi
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: postgresql-pgvector
spec:
  replicas: 1
  strategy:
    type: Recreate          # RW0-levyä ei voi jakaa kahdelle podille
  selector:
    matchLabels: { app: postgresql-pgvector }
  template:
    metadata:
      labels: { app: postgresql-pgvector }
    spec:
      securityContext: {}    # EI runAsUser
      containers:
        - name: postgresql
          image: quay.io/rh-aieservices-bu/postgresql-15-pgvector-c9s
          envFrom:
            - secretRef: { name: postgres-secret }
          ports:
            - containerPort: 5432
          volumeMounts:
            - name: data
              mountPath: /var/lib/pgsql/data
          resources:
            requests: { cpu: 100m, memory: 512Mi }
            limits:   { cpu: 500m, memory: 1Gi }
      volumes:
        - name: data
          persistentVolumeClaim:
            claimName: pgvector-data

```

```
---
apiVersion: v1
kind: Service
metadata:
  name: postgresql-pgvector
spec:
  selector: { app: postgresql-pgvector }
  ports:
    - port: 5432
      targetPort: 5432
```

`strategy: Recreate` on tärkeä: oletuksena rullaava päivitys yrittäisi käynnistää uuden podin ennen vanhan sammuttamista, mutta RWO-levyä ei voi liittää kahteen podiin yhtä aikaa — uusi podi jää `Pending`-tilaan.

pgvector-laajennuksen käyttöönotto

Laajennus otetaan käyttöön kerran, tietokannan sisällä:

```
CREATE EXTENSION vector;
```

Suoraan podista:

```
oc exec -it deployment/postgresql-pgvector -n <projekti> -- \
  psql -U postgres -d vectordb -c "CREATE EXTENSION IF NOT EXISTS vector;"
```

Tai pgAdminissa: *Query Tool* (Ctrl+Shift+Q) → sama komento → F5. Vaihtoehtoisesti napsauta hiiren oikealla *Extensions* → *Create* → *Extension...* ja valitse `vector`.

Käyttöesimerkki:

```
CREATE TABLE items (id bigserial PRIMARY KEY, embedding vector(3));
INSERT INTO items (embedding) VALUES ('[1,2,3]'), ('[4,5,6]');

-- Lähimmät naapurit (L2-etäisyys)
SELECT * FROM items ORDER BY embedding <-> '[3,1,2]' LIMIT 5;
```

Isommilla aineistoilla tarvitet indeksin:

```
CREATE INDEX ON items USING hnsw (embedding vector_cosine_ops);
```

Yhteys sovelluksesta

Saman projektin sisällä tietokanta löytyy palvelunimellä:

```
DATABASE_URL=postgresql://postgres:<salasana>@postgresql-pgvector:5432/vectordb
```

```
oc create secret generic sovellus-db \
  --from-literal=DATABASE_URL='postgresql://postgres:<salasana>@postgresql-
pgvector:5432/vectordb' \
  -n <projekti>
oc set env deployment/sovellus --from=secret/sovellus-db -n <projekti>
```

Paikallinen hallinta port-forwardilla

Koska Routea ei ole, tietokantaa hallitaan tunneloimalla:

```
oc project <projekti>
oc port-forward svc/postgresql-pgvector 5432:5432
```

Jätä ikkuna auki. Yhdistä pgAdminilla tai `psql` :llä:

Kenttä	Arvo
Host	localhost
Port	5432
Database	vectordb
Username	postgres
Password	POSTGRESQL_PASSWORD

```
psql postgresql://postgres:<salasana>@localhost:5432/vectordb
```

Port-forward on tilapäinen hallintayhteys — ei tapa, jolla sovellus kutsuu tietokantaa.

Varmuuskopiot

Varmuuskopiointi on **sinun vastuullasi**. Minimitaso:

```
# Dumppaa paikalliseen tiedostoon
oc exec deployment/postgresql-pgvector -n <projekti> -- \
  pg_dump -U postgres vectordb > backup-$(date +%F).sql

# Palautus
cat backup-2026-08-31.sql | oc exec -i deployment/postgresql-pgvector -n <projekti> -- \
  psql -U postgres -d vectordb
```

Dumpit kannattaa viedä Allakseen: [8. Allas S3 -tallennus](#).

8. Allas-objektitallennus (S3)

Rahdin podit ovat tilattomia: podin levyille kirjoitettu data katoaa. Pysyvä data kuuluu joko PVC:lle tai — isot tiedostot, jaettu käyttö, varmuuskopiot — CSC:n Allas-objektitallennukseen.

Sisällys

- S3 vai Swift
- 1. Luo bucket
- 2. Asenna OpenStack-työkalut
- 3. Hae S3-tunnukset
- 4. Tallenna tunnukset
- 5. Käyttö koodista
- Tunnukset Rahtiin
- Vianmääritys

S3 vai Swift

Allas tukee kahta protokollaa. **Käytä S3:a sovelluksissa.**

	S3	Swift
Tunnusten elinikä	pitkäikäiset avaimet	token vanhenee ~8 h
Sopii Rahtiin	kyllä	huonosti — token vanhenisi kesken ajon
Kirjastotuki	boto3, AWS SDK, s3cmd, rclone	swift-client

Älä sekoita protokollia samassa bucketissa.

Palvelu	Endpoint	Region
S3 API	<code>https://a3s.fi</code>	<code>regionOne</code>
Swift API	<code>https://a3s.fi:443/swift/v1/AUTH_<project-id></code>	<code>regionOne</code>

Region on aina `regionOne` , vaikka palvelin on Suomessa. Ei `eu-north-1` .

1. Luo bucket

1. Kirjaudu allas.csc.fi
2. Valitse oikea projekti vasemmalta
3. + **Create bucket**

Nimeäminen: bucket-nimien on oltava uniikkeja **kaikkien Allas-käyttäjien kesken**. CSC suosittelee etuliitteeksi projektin numeroa, esim. `1234567-raw-data`. Vain pieniä kirjaimia, numeroita ja väliviivoja — ei ääkkösiä.

Kolme eri "projektia" — älä sekoita niitä. OpenStackin `project_XXXXXXX` on sama **CSC-laskentaprojekti**, jonka numeron annoit Rahti-projektille kentässä `csc_project` (luku 1). Rahti-projekti eli namespace on eri asia: se on laskentaprojektin sisällä oleva nimiavaruus.

Tunnukset ja bucketit ovat laskentaprojektikohtaisia. Varmista että luot tunnukset samassa projektissa, jossa bucket on — muuten saat 403-virheen etkä ymmärrä miksi.

2. Asenna OpenStack-työkalut

Tunnukset haetaan OpenStack-komentorivityökalulla.

```
python -m pip install --user python-openstackclient
```

Jos `openstack` -komento ei löydy asennuksen jälkeen, Scripts-kansio ei ole PATHissa. Selvitä oikea polku:

```
python -c "import sys, os; print(os.path.dirname(sys.executable) + '\\Scripts')"
```

Lisää tuloste PATHiin (pysyvästi: Windows + R → `sysdm.cpl` → *Advanced* → *Environment Variables* → *Path* → *New*), avaa uusi terminaali ja tarkista:

```
openstack --version
```

Konfiguraatio:

1. Kirjaudu pouta.csc.fi
2. **API Access** → **Download OpenStack RC File** → **OpenStack clouds.yaml file**
3. Tallenna tiedosto polkuun `~/.config/openstack/clouds.yaml` (Windowsilla `C:\Users\<tunnus>\.config\openstack\clouds.yaml`)

```
mkdir "$HOME\.config\openstack"
```

Tiedoston sisältö on tätä muotoa:

```
clouds:
  openstack:
    auth:
      auth_url: https://pouta.csc.fi:5001/v3
      username: "<csc-tunnus>"
      project_id: "<projektin-id>"
      project_name: "project_XXXXXXX"
      user_domain_name: "Default"
      # password: jätä pois – kysytään ajettaessa
    regions:
      - regionOne
    interface: "public"
    identity_api_version: 3
```

3. Hae S3-tunnukset

```
$env:OS_CLOUD = "openstack"

# Varmista että olet oikeassa projektissa
openstack configuration show          # katso auth.project_id
openstack project list                # kaikki projektit
openstack project set project_XXXXXX # vaihda tarvittaessa

# Listaa olemassa olevat tunnukset
openstack ec2 credentials list

# Luo uudet, jos tarvitaan
openstack ec2 credentials create
```

Komento kysyy CSC-salasanaa. Tuloste:

```

+-----+-----+-----+
-----+-----+
| Access                               | Secret                               | Project ID
| User ID |
+-----+-----+-----+
-----+-----+
| abc123def456ghi789jkl012mno345pq | xyz789uvw456rst123qpo890lmn567abc |
def456abc789ghi012jkl345mno678pq | tunnus |
+-----+-----+-----+
-----+-----+

```

`Access` = access key, `Secret` = secret key.

CSC:n supertietokoneilla (Roihu, LUMI) sama onnistuu yhdellä komennolla: `allas-conf` näyttää S3-yhteystiedot suoraan.

4. Tallenna tunnukset

Paikallisesti `.env` -tiedostoon (joka on `.gitignore` ssa):

```

ALLAS_ACCESS_KEY_ID=abc123def456ghi789jkl012mno345pq
ALLAS_SECRET_ACCESS_KEY=xyz789uvw456rst123qpo890lmn567abc
ALLAS_ENDPOINT_URL=https://a3s.fi
ALLAS_BUCKET_NAME=1234567-raw-data

```

5. Käyttö koodista

Tärkein yksityiskohta: Allas tukee vain **path-style**-osoitteita (`a3s.fi/<bucket>/<avain>`), ei virtual-host-tyyliä (`<bucket>.a3s.fi`). AWS SDK v3 käyttää oletuksena virtual-hostia, joten asetus on pakko ohittaa.

Python (boto3)

```
import boto3, os

s3 = boto3.client(
    "s3",
    endpoint_url=os.getenv("ALLAS_ENDPOINT_URL"),
    aws_access_key_id=os.getenv("ALLAS_ACCESS_KEY_ID"),
    aws_secret_access_key=os.getenv("ALLAS_SECRET_ACCESS_KEY"),
    region_name="regionOne",
)

for b in s3.list_buckets()["Buckets"]:
    print(b["Name"])

bucket = os.getenv("ALLAS_BUCKET_NAME")
s3.upload_file("paikallinen.txt", bucket, "kansio/etä.txt")
s3.download_file(bucket, "kansio/etä.txt", "ladattu.txt")
```

boto3 osaa path-stylen automaattisesti tälle endpointille. Jos törmäät ongelmiin, pakota se:

```
from botocore.config import Config
s3 = boto3.client("s3", ..., config=Config(s3={"addressing_style": "path"}))
```

JavaScript / TypeScript (AWS SDK v3)

```
import { S3Client, ListBucketsCommand, PutObjectCommand } from "@aws-sdk/client-s3";

const s3 = new S3Client({
  endpoint: process.env.ALLAS_ENDPOINT_URL, // https://a3s.fi
  region: "regionOne",
  forcePathStyle: true, // ← PAKOLLINEN Allakselle
  credentials: {
    accessKeyId: process.env.ALLAS_ACCESS_KEY_ID,
    secretAccessKey: process.env.ALLAS_SECRET_ACCESS_KEY,
  },
});

const { Buckets } = await s3.send(new ListBucketsCommand({}));

await s3.send(
  new PutObjectCommand({
    Bucket: process.env.ALLAS_BUCKET_NAME,
    Key: "test.txt",
    Body: "Hei Allas!",
  })
);
```

Tunnukset Rahtiin

```
oc create secret generic allas-credentials \
  --from-literal=ALLAS_ACCESS_KEY_ID='...' \
  --from-literal=ALLAS_SECRET_ACCESS_KEY='...' \
  --from-literal=ALLAS_ENDPOINT_URL='https://a3s.fi' \
  --from-literal=ALLAS_BUCKET_NAME='1234567-raw-data' \
  -n <projekti>

oc set env deployment/sovellus --from=secret/allas-credentials -n <projekti>
```

Nämä ovat ajonaikaisia muuttujia — **älä laita niitä** `NEXT_PUBLIC_` - tai `VITE_` -etuliitteen taakse, muuten avaimet päätyvät selaimeen (ks. [05 Ympäristömuuttujat](#)).

Jos build kaatuu puuttuviin avaimiin, anna sille tyhjä paikanpitäjä:

```
ARG ALLAS_ACCESS_KEY_ID=""  
ENV ALLAS_ACCESS_KEY_ID=$ALLAS_ACCESS_KEY_ID  
RUN npm run build
```

Vianmäärittäminen

Oire	Todennäköinen syy
<code>SignatureDoesNotMatch</code>	Väärä secret key, tai virtual-host-osoitteet päällä → <code>forcePathStyle: true</code>
<code>403 Forbidden</code> bucketiin	Tunnukset luotu eri projektissa kuin bucket
<code>NoSuchBucket</code>	Bucket on toisessa projektissa, tai nimessä kirjoitusvirhe
<code>openstack: command not found</code>	Python Scripts -kansio ei ole PATHissa (ks. yllä)
<code>pip: command not found</code>	Käytä <code>python -m pip install ...</code>
Yhteys aikakatkeaa Rahdista	Tarkista ettei endpointissa ole <code>http://</code> — vain <code>https://a3s.fi</code>

Edellinen: [7. Tietokanta](#) · **Seuraava:** [9. Vianmäärittäminen](#) →

Lähteet: [CSC: Allas](#) · [CSC: How to get Allas S3 credentials](#)

9. Vianmääritys

Oireesta syhyhyn. Aloita aina samasta kolmesta komennosta, älä arvaa.

Sisällys

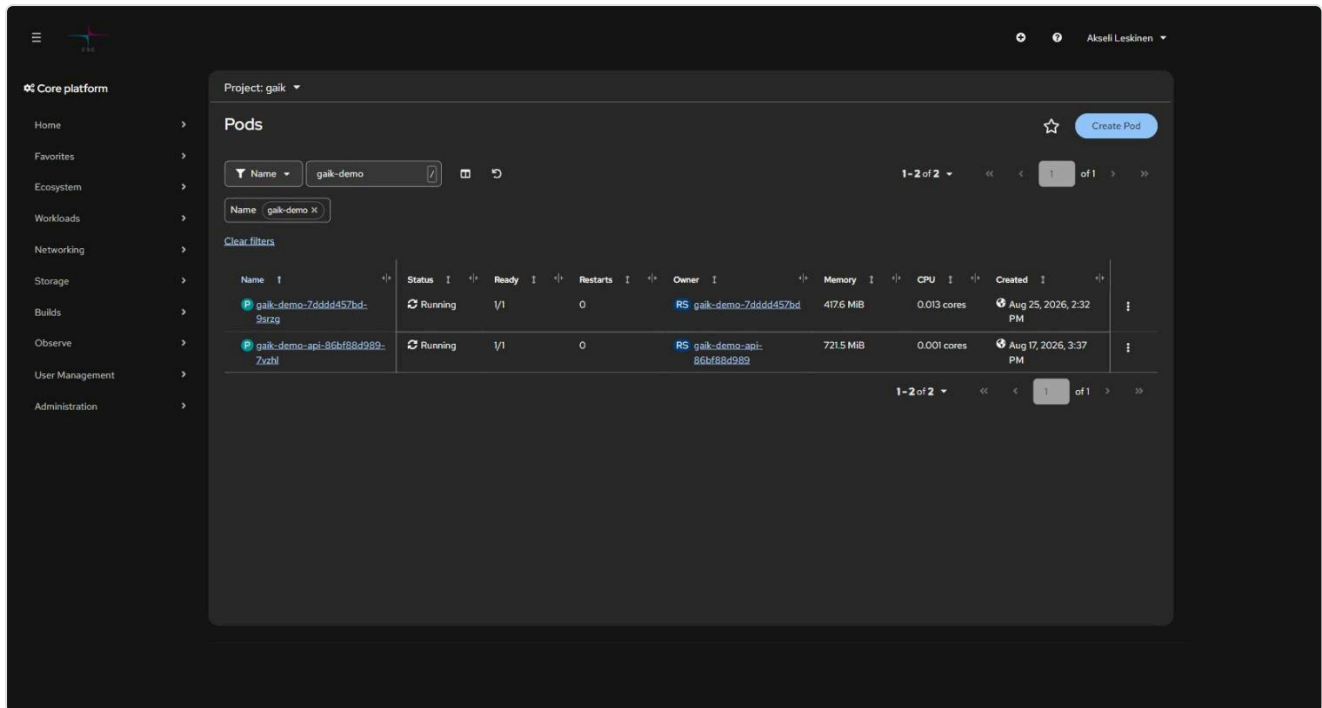
- Ensimmäiset kolme komentoa
- Podin tilat ja mitä ne tarkoittavat
- Deployment ei päivity pushin jälkeen
- CrashLoopBackOff
- ImagePullBackOff / ErrImagePull
- Pod jää Pending-tilaan
- OOMKilled (exit 137)
- Permission denied tiedostoa kirjoitettaessa
- 500-virhe docker pushissa
- 504 Gateway Time-out
- Sovellus ei vastaa Routen kautta
- Unauthorized / kirjautuminen vanhentunut
- Vääriä hälytyksiä
- Koko namespacen tilannekuva

Ensimmäiset kolme komentoa

```
oc get pods -n <projekti> # 1. mikä on tila
oc logs deployment/<sovellus> -n <projekti> --tail=50 # 2. mitä sovellus sanoo
oc get events -n <projekti> --sort-by=.lastTimestamp | tail # 3. mitä klusteri sanoo
```

Podin tila kertoo, kumpaan suuntaan mennä: `Running` mutta väärä käytös → logit.

`Pending` / `CrashLoop` / `ImagePull` → eventit ja `oc describe`.



Samat tiedot löytyvät konsolista: *Workloads* → *Pods* → *<pod>* → *Logs / Events*.

Podin tilat ja mitä ne tarkoittavat

Tila	Merkitys	Seuraava komento
Running + 1/1	Käynnissä ja valmis	—
Running + 0/1	Käynnissä, mutta readiness-tarkistus ei mene läpi	<code>oc describe pod <pod></code>
Pending	Ei skeduloitu — kiintiö, PVC tai resurssipyyntö	<code>oc describe pod <pod></code>
CrashLoopBackOff	Käynnistyy ja kaatuu toistuvasti	<code>oc logs <pod> --previous</code>
ImagePullBackOff	Imagea ei saada haettua	<code>oc describe pod <pod></code>
Error / Completed	Prosessi päättyi (job, build)	Normaalia jobeille
Terminating pitkään	Sammutus jumissa	<code>oc delete pod <pod> --force</code>

Deployment ei päivity pushin jälkeen

Yleisin hämmennyksen aihe: image on työnnetty, mutta vanha koodi pyörii yhä.

```
# 1. Tuliko uusi image perille?
oc get is <imagestream> -n <projekti>
oc describe is <imagestream> -n <projekti> | head -30

# 2. Onko image trigger olemassa ja päällä?
oc get deployment <sovellus> -n <projekti> \
  -o jsonpath="{.metadata.annotations.image\.openshift\.io/triggers}"
```

Jos tuloste on tyhjä tai sisältää `"paused": "true"`:

```
oc annotate deployment/<sovellus> image.openshift.io/triggers- -n <projekti>
oc set triggers deployment/<sovellus> \
  --from-image=<imagestream>:latest -c <kontti> -n <projekti>
```

Pikakorjaus ilman triggeriä:

```
oc rollout restart deployment/<sovellus> -n <projekti>
oc rollout status deployment/<sovellus> -n <projekti>
```

Tai aseta image käsin:

```
LATEST=$(oc get is <imagestream> -n <projekti> \
  -o jsonpath='{.status.tags[0].items[0].dockerImageReference}')
oc set image deployment/<sovellus> <kontti>=${LATEST} -n <projekti>
```

Tarkista myös buildit, jos käytät BuildConfigia — epäonnistunut buildi tarkoittaa, ettei uutta imagea koskaan syntynyt:

```
oc get builds -n <projekti>
```

CrashLoopBackOff

```
oc logs deployment/<sovellus> -n <projekti> --previous # edellisen yrityksen logi
oc describe pod <pod> -n <projekti> # exit code ja eventit
```

Tavallisimmat syyt:

Syy	Tunnusmerkki
Puuttuva ympäristömuuttuja	Logissa <code>undefined</code> , <code>KeyError</code> , <code>connection string is empty</code>
Sovellus kuuntelee <code>127.0.0.1</code> :tä	Käynnistyy normaalisti, mutta terveystarkistus ei vastaa
Väärä portti	<code>containerPort</code> $\neq$ sovelluksen portti
Root-oikeuksien tarve	<code>permission denied</code> , <code>EACCES</code> , <code>mkdir failed</code>
Väärä arkkitehtuuri	<code>exec format error</code> → image on arm64, tarvitaan amd64
OOM	Exit code 137

ImagePullBackOff / ErrImagePull

```
oc describe pod <pod> -n <projekti> | grep -A5 Events
```

- **Yksityinen rekisteri ilman pull secretiä** → luo secret ja linkitä se:

```
oc secrets link default <pull-secret> --for=pull -n <projekti>
```

- **Väärä tagi** → tarkista, että tagi on olemassa: `oc get is <nimi> -o yaml`
- **Väärä osoite** → klusterin sisällä `image-registry.openshift-image-registry.svc:5000/<projekti>/<image>`, ulkopuolelta `image-registry.apps.2.rahti.csc.fi/<projekti>/<image>`

Pod jää Pending-tilaan

```
oc describe pod <pod> -n <projekti> | tail -20
oc describe AppliedClusterResourceQuotas
```

Kolme tyypillistä syytä:

1. **Kiintiö täynnä.** Kiintiö on jaettu koko laskentaprojektin kesken. Sammuta tarpeettomat sovellukset tai skaalaa nolnaan: `oc scale deployment/<x> --replicas=0`
2. **PVC ei saa levyä.** `oc get pvc -n <projekti>` — jos tila on `Pending`, koko voi ylittää kiintiön (max 100 GiB / PVC).
3. **Resurssipyyntö ylittää rajan.** LimitRange määrää maksimit; liian iso `requests` hylätään.

OOMKilled (exit 137)

Kontti ylitti muistirajansa.

```
oc adm top pods -n <projekti>
oc set resources deployment/<sovellus> -n <projekti> --limits=memory=2Gi --
requests=memory=512Mi
```

Muista suhderajoitus: `limits` saa olla korkeintaan $5 \times$ `requests` (tarkista projektisi LimitRange). Pelkän katon nostaminen ilman varauksen nostoa voi siis hylkääntyä.

Permission denied tiedostoa kirjoitettaessa

Rahti antaa kontille satunnaisen UID:n, mutta ryhmä on aina **0**. Tee kirjoitettavista hakemistoista ryhmän 0 kirjoitettavia:

```
RUN mkdir -p /app/output && \
  chown -R 1001:0 /app/output && \
  chmod -R g+rwX /app/output
USER 1001
```

Sama koskee `.next` -, `tmp` - ja `cache`-hakemistoja. Jos sovellus kirjoittaa juurihakemistoon, ohjaa se kirjoittamaan `/tmp`:hen — se on aina kirjoitettavissa.

Älä yritä korjata tätä `runAsUser` -asetuksella manifestissa; SCC hylkää sen.

500-virhe docker pushissa

```
unknown: unexpected status from HEAD request to
https://image-registry.apps.2.rahti.csc.fi/v2/<projekti>/<image>/manifests/sha256:... : 500
```

ImageStream puuttuu. Luo se ensin:

```
oc create imagestream <image> -n <projekti>
```

Tarkista myös, että olet kirjautunut rekisteriin (`oc whoami -t` -tokenilla) ja että image on alle 5 GiB.

504 Gateway Time-out

HAProxyn oletustimeout on 30 sekuntia. Ks. [04 Reitit ja verkko](#).

```
oc annotate route <reitti> haproxy.router.openshift.io/timeout=120s -n <projekti> --
overwrite
```

Sovellus ei vastaa Routen kautta

Etene ulkoa sisäänpäin:

```
# 1. Onko Route olemassa ja hyväksytty?
oc get route <reitti> -n <projekti>

# 2. Osoittaako se oikeaan palveluun ja porttiin?
oc describe route <reitti> -n <projekti>

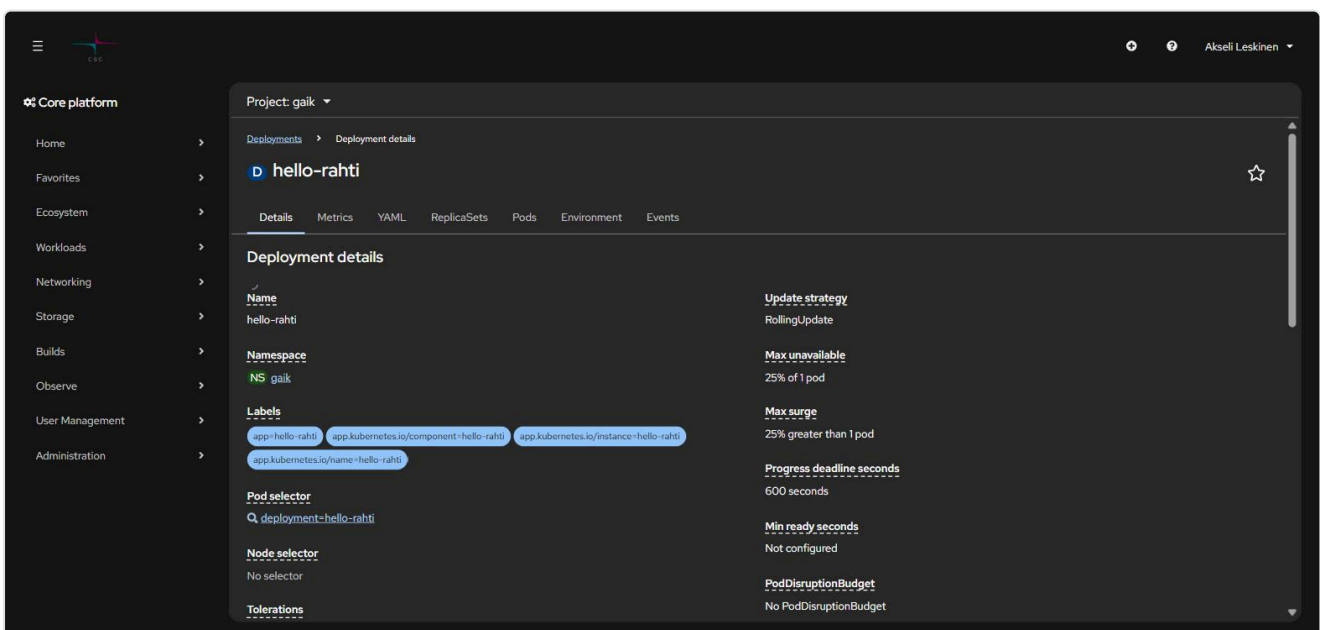
# 3. Vastaako Service?
oc get svc <palvelu> -n <projekti> -o wide
oc get endpoints <palvelu> -n <projekti>      # tyhjä = label-valitsin ei osu podeihin

# 4. Vastaako podi suoraan?
oc port-forward deployment/<sovellus> 8080:<portti> -n <projekti>
curl -I http://localhost:8080
```

Tyhjä `endpoints` -lista on erittäin yleinen: Servicen `selector` ei täsmää podin labeleihin.

Klassinen tapaus: `oc new-app` merkitsee podit labelilla `deployment=<nimi>`, ei `app=<nimi>`. Siksi `oc get pods -l app=<nimi>` palauttaa "No resources found" vaikka podi on pystyssä, ja käsin kirjoitettu `selector: {app: <nimi>}` jättää Servicen tyhjäksi. Tarkista todelliset labelit ennen kuin arvaat:

```
oc get pods -n <projekti> --show-labels
oc get svc <palvelu> -n <projekti> -o jsonpath='{.spec.selector}'
```



Konsolissa sama näkyy Deploymentin *Details*-välilehdellä kohdassa **Pod selector**.

Unauthorized / kirjautuminen vanhentunut

```
error: You must be logged in to the server (Unauthorized)
```

Henkilökohtainen token vanhenee noin vuorokaudessa. Tarkista ensin kuka olet:

```
oc whoami  
oc config current-context
```

Jos konteksti on vaihtunut takaisin henkilötunnukseen vaikka käytössä pitäisi olla service account, kirjaudu sillä uudelleen. Pysyvä ratkaisu on service account pitkäikäisellä tokenilla: [1. Aloitus](#).

Väriä hälytyksiä

Nämä näyttävät vioilta mutta eivät ole:

- **Restart-luku ei yksin ole hälytys.** Kuukausia ajanut podi kerää restartteja normaalista kierrätyksestä. Suhteuta se podin ikään: 40 restarttia 90 päivässä on eri asia kuin 40 restarttia tunnissa.
- **HTTP 401/403 Routella tarkoittaa, että sovellus vastaa.** Se on pystyssä ja vaatii kirjautumisen. Vain 5xx, yhteysvirhe ja timeout ovat hälytyksiä.
- **Ensimmäinen pyyntö vähän käytettyyn sovellukseen voi aikakatkaista** (kylmä käynnistys). Kokeile toisen kerran ennen kuin merkitset alhaalla olevaksi.
- **Completed -tilaiset podit eivät ole vikoja** — ne ovat päätyneitä job-, cronjob- ja build-ajoja.
- **Tyhjä resourcequota** on normaali: kiintiö on laskentaprojektin tasolla. Käytä `oc describe AppliedClusterResourceQuotas`.
- **Elävässä Deploymentissa näkyy @sha256:... eikä :latest** — image trigger on ratkaissut tagin digestiksi. Näin sen kuuluukin toimia.

Koko namespaces tilannekuva

Kun haluat tietää kerralla, onko kaikki pystyssä:

```
oc get all -n <projekti>  
oc get events -n <projekti> --field-selector type=Warning --sort-by=.lastTimestamp
```

Tämän repositorion mukana tulee myös **rahti-audit -skilli**, joka kerää saman tilannekuvan yhdellä ajolla (deploymentit, podit, routet HTTP-vasteineen, warning-eventit ja kiintiöt) ja raportoi vain poikkeamat. Ks. [10. Agenttinen kehitys](#).

Edellinen: [8. Allas S3](#) · **Seuraava:** [10. Agenttinen kehitys](#) →

10. Agenttinen kehitys: skillit Rahti-työhön

Tämä repositorio sisältää neljä **agenttiskilliä**, jotka opettavat tekoälyagentin (Claude Code, Codex, Cursor tms.) käyttämään CSC:n ympäristöä oikein. Skilli on pelkkä markdown-tiedosto — ei asennettavaa ohjelmistoa, ei API-avainta.

Sisällys

- Mikä skilli on
- Repositorion skillit
- Asennus
 - Vähemmän lupakyselyitä
- Käyttö
- UI vai CLI vai agentti
- Mitä agentti ei voi tehdä
- Turvasäännöt
- Skillien muokkaus
- Skillit useammalla koneella
- Lue lisää

Mikä skilli on

Skilli on kansio, jossa on `SKILL.md` ja mahdollisia lisätiedostoja:

```
csc-rahti/
├── SKILL.md           # ohjeet agentille + kuvaus milloin skilli otetaan käyttöön
├── references/        # syventävät dokumentit, luetaan vain tarvittaessa
│   ├── routes.md
│   ├── authentication.md
│   └── ...
├── scripts/           # ajettavat apuskriptit
└── tests/             # skriptien testit
```

`SKILL.md`:n alussa on YAML-lohko, jonka `description` -kenttä kertoo agentille **milloin** skilli kannattaa ottaa käyttöön. Agentti lukee kuvaukset jatkuvasti, mutta koko sisällön vasta kun se on relevantti — näin kymmenetkään skillit eivät täytä konteksti-ikkunaa.

Käytännön hyöty: ilman skilliä agentti arvaa Rahdin yksityiskohdat väärin (vanhentunut konsolin navigaatio, `runAsUser` manifestissa, unohtunut `ImageStream`). Skillin kanssa se tietää talon tavat.

Repositorion skillit

Skilli	Mihin	Milloin laukeaa
csc-rahti	Rahti-deployaus, imaget, routet, salaisuudet, Satama, Allas, Aitta-LLM-API	"deployaa Rahtiin", "oc", "rahtiapp.fi", "imagestream", "satama"
rahti-audit	Read-only tilannekuva koko namespacesta	"onko tuotanto kunnossa", "auditoi Rahti", "crashloop"
csc-roihu	Roihu-supertietokone: Slurm, GH200-GPU:t, moduulit	"sbatch", "aja GPU:lla", "supertietokone"
csc-lumi	LUMI: AMD MI250X, ROCm, LUMI-ohjelmistopino	"LUMI", "ROCm", "MI250X"

Työnjako on tarkoituksellinen: **rahti-audit** on **read-only** eikä saa korjata mitään, **csc-rahti** tekee muutokset. Näin "katso mikä on rikki" ei koskaan vahingossa muuta tuotantoa.

Valinta CSC-palveluiden välillä lyhyesti:

Tarve	Palvelu	Skilli
Web-sovellus tai API pystyy	Rahti	csc-rahti
Mallin koulutus tai iso eräajo NVIDIA-GPU:lla	Roihu	csc-roihu
Sama AMD-GPU:lla, EuroHPC-kiintiöllä	LUMI	csc-lumi
Valmis LLM-päätepine ilman omaa GPU:ta	Aitta	csc-rahti (Aitta-osio)

Asennus

Vaihtoehto 1: kloonaa repositorio (helpoin)

Kun avaat tämän repositorion Claude Codella, skillit ovat heti käytössä — ne ovat kansiossa `.claude/skills/`, jonka Claude Code lukee projektikohtaisesti.

```
git clone <tämän-repon-url>
cd csc-rahti-guide
claude
```

Vaihtoehto 2: kopioi omaan käyttöön (kaikkiin projekteihin)

Kopioi haluamasi skillit henkilökohtaiseen skillikansioosi, niin ne ovat käytössä kaikissa projekteissa:

```
# macOS / Linux
cp -r .claude/skills/csc-rahti ~/.claude/skills/
cp -r .claude/skills/rahti-audit ~/.claude/skills/
```

```
# Windows PowerShell
Copy-Item -Recurse .claude\skills\csc-rahti "$HOME\.claude\skills\"
Copy-Item -Recurse .claude\skills\rahti-audit "$HOME\.claude\skills\"
```

Tarkista tulos Claude Codessa komennolla `/skills`.

Vaihtoehto 3: jaettu `.agents/skills` -kansio

Osa agenttityökaluista lukee skillit polusta `.agents/skills/` (agentskills.io-konventio). Sama `SKILL.md` kelpaa molemmille, joten tiedostoja ei kannata monistaa — tee linkki:

```
# macOS / Linux
mkdir -p ~/.agents/skills
ln -s ~/.claude/skills/csc-rahti ~/.agents/skills/csc-rahti
```

```
# Windows: hakemistojunktio (ei vaadi järjestelmänvalvojan oikeuksia)
New-Item -ItemType Junction -Path "$HOME\.agents\skills\csc-rahti" `
    -Target "$HOME\.claude\skills\csc-rahti"
```

Claude Code ei lue `.agents/skills` -polkua — sille riittää `~/.claude/skills/` tai projektin `.claude/skills/`. Linkki on siis muita työkaluja varten, ja tehdään siihen suuntaan että Claude Coden polku on alkuperäinen ja `.agents` osoittaa siihen.

Jos työkalusi lukee ohjeensa `AGENTS.md` -tiedostosta, lisää sinne rivi, joka viittaa tämän repositorion `docs/` -kansioon ja skilleihin.

Vähemmän lupakyselyitä

Repossa on `.claude/settings.json`, joka sallii valmiiksi vain lukevat `oc` -komennot (`get`, `describe`, `logs`, `status`, `whoami`). Agentti voi siis tutkia tilannetta kysymättä joka kerta lupaa, mutta **jokainen kirjoittava komento** — `apply`, `delete`, `scale`, `set`, `rollout` — kysyy edelleen erikseen. Halutessasi voit poistaa tiedoston tai laajentaa listaa omaan makuusi.

Riippuvuudet

Skillit itsessään eivät vaadi mitään, mutta niiden neuvomat komennot vaativat:

Työkalu	Mihin	Tarkistus
<code>oc</code>	kaikki Rahti-toiminta	<code>oc version --client</code>
<code>docker</code> tai <code>podman</code>	imagejen rakennus	<code>docker --version</code>
PowerShell 7	<code>csc-rahti</code> - ja <code>rahti-audit</code> -skriptit	<code>pwsh --version</code>
<code>openstack</code>	Allas-tunnukset	<code>openstack --version</code>

Käyttö

Kutsu nimellä (Claude Code, slash-komento):

```
/csc-rahti Miten saan omalle sovellukselle osoitteen demo.2.rahtiapp.fi?
```

Tai kuvaile tehtävä — skilli laukeaa kuvauksensa perusteella itsestään:

```
Deployaa tämä Next.js-sovellus Rahtiin projektiin minun-projekti,
portti 3000, ja tee sille route.
```

```
Tarkista onko Rahdissa mitään rikki.
```

```
gaik-demo palauttaa 504 kun kysely kestää yli puoli minuuttia. Korjaa.
```

Tyypillinen agenttityönkulku Rahtiin:

1. **Kirjaudu itse** — agentti ei läpäise MFA:ta (ks. alla). Aja `oc login` tai `Connect-Rahti.ps1`.
2. **Anna tehtävä.** Agentti lukee skillin, tarkistaa tilanteen `oc get` -komennoilla ja ehdottaa muutokset.
3. **Tarkista ehdotus** ennen kuin hyväksyt kirjoittavat komennot.
4. **Anna sen ajaa deploy** ja lukea `oc rollout status` -tuloste.

UI vai CLI vai agentti

Tehtävä	Suositus	Miksi
Ensimmäinen tutustuminen	UI	Näet mitä objekteja syntyy
Kertaluontoinen demo pystyyn	UI	Nopein, ei tiedostoja
Toistuva deployaus	CLI + manifestit	Versionhallinnassa, toistettavissa
Ympäristömuuttujan pikamuutos	UI tai <code>oc set env</code>	Molemmat käynnistävät rullauksen
Salaisuuksien luonti	CLI	<code>--from-literal</code> on tarkka; lomake ei näytä lopputulosta
Lokien lukeminen	UI kevyeen, <code>oc logs -f</code> vakavaan	UI katkaisee pitkät lokit
Tuotannon tilannekuva	Agentti (<code>rahti-audit</code>)	Kymmenen komentoa yhdellä ajolla, raportoi vain poikkeamat
Vianmääritys	Agentti + CLI	Agentti osaa etenemisjärjestyksen, sinä päätät korjauksen
Kiintiöiden nosto, oikeudet	UI + palvelupiste	Vaatii ihmisen

Nyrkkisääntö: **UI oppimiseen ja kertaluontoiseen, CLI toistettavaan, agentti rutiinien nopeuttamiseen.** Agentti ei korvaa sitä, että ymmärrät mitä se tekee — siksi luvut 1–9 kannattaa lukea vaikka antaisit agentin tehdä työn.

Mitä agentti ei voi tehdä

- **Läpäistä CSC:n MFA:ta.** Henkilökohtainen kirjautuminen on aina ihmisen tehtävä. Agentti voi käyttää valmiiksi luotua service accountia, mutta ei luoda ensimmäistä tunnusta ilman että olet kirjautunut.
- **Hakea tokenia web-konsolista.** *Copy login command* vaatii selainistunnon.
- **Nostaa kiintiötä tai avata Rahti-oikeuksia** — ne kulkevat MyCSC:n ja palvelupisteen kautta.
- **Tietää projektikohtaisia rajoja arvaamatta.** Anna sen ajaa `oc describe limitranges` sen sijaan että luottaisit oletusarvoihin.

Turvasäännöt

Skillit on kirjoitettu näiden sääntöjen mukaan, ja samat pätevät sinuun:

1. **Tokeneita ei tulosteta.** Ei chattiin, ei lokiin, ei committiin. Skriptit välittävät tokenin suoraan `oc :lle`.

2. **Salaisuudet repositorion ulkopuolelle.** Service account -tokenit tallennetaan erilliseen env-hakemistoon, ei skilliin eikä projektiin.
3. **Read-only pysyy read-only.** rahti-audit ei aja `oc delete`, `oc scale`, `oc apply` -komentoja edes silloin kun korjaus näyttää itsestään selvältä.
4. **Kirjoittavat komennot tuotantoon vahvistetaan.** `oc delete`, `oc scale --replicas=0` ja `oc apply` ovat asioita, jotka kannattaa lukea ennen hyväksymistä.
5. **Vähimmät oikeudet.** `view` auditointiin, `edit` deployaukseen, `admin` vain jos oikeuksien hallinta on oikeasti tarpeen.

Skillien muokkaus

Skillit ovat tarkoituksella luettavaa markdownia — muokkaa niitä omaan ympäristöösi:

- Vaihda `<namespace>` -paikanpitäjät oman projektisi nimeen, jos työskentelet vain yhdessä.
- Lisää `references/` -kansioon oman organisaatiosi käytännöt.
- Pidä `description` -kenttä kuvaavana: se ratkaisee, laukeaako skilli oikeaan aikaan.
- Skriptit ovat testattuja — jos muutat niitä, aja testit:

```
pwsh -NoProfile -Command "& '.\claude\skills\csc-rahti\tests\RahtiCredentials.Tests.ps1'"
```

Älä committaa skilliin projektikohtaisia namespaceja, tunnuksia, asiakkaiden nimiä tai tokeneita, jos repositorio on julkinen.

Skillit useammalla koneella

Jos käytät useampaa konetta, älä kopioi skillejä koneelta toiselle — ne eriytyvät viikossa. Pidä yksi kansio ainoana lähteenä ja linkitä siihen:

```
# Kansio joka synkronoituu koneiden välillä (OneDrive, Dropbox, git-repo...)
$src = "$HOME\OneDrive\agents-setup\skills"

# Claude Code lukee tämän polun
New-Item -ItemType Junction -Path "$HOME\claude\skills" -Target $src
```

```
# macOS / Linux
ln -s ~/Sync/agents-setup/skills ~/.claude/skills
```

Sama kansio voi palvella useaa työkalua yhtä aikaa: yksi lähde, monta linkkiä. Salaisuudet eivät kuulu tähän kansioon — pidä ne erillään (esim. `agents-setup/env/`), jotta skillien jakaminen ei koskaan jaa tokeneita.

Git-repo synkronointikansiona on paras vaihtoehto tiimille: muutokset ovat katselmoitavissa ja historia näkyy.

Lue lisää

Skillit yleisesti

- [What are Skills?](#) — Anthropicin yleiskuvaus
- [Claude Code: Skills](#) — `SKILL.md` -formaatti, frontmatter-kentät ja hakupolut
- [agentskills.io](#) — työkaluriippumaton spesifikaatio, jota tämän repon skillit noudattavat

Skillien paketointi laajempaan jakeluun

Jos skillejä alkaa kertyä ja niitä halutaan jakaa organisaatiossa, seuraava askel on paketoida ne pluginiksi — skillit, työkalut ja konfiguraatio yhtenä asennettavana kokonaisuutena:

- [Agent plugins: package your skills, tools and more](#) — mitä plugin sisältää ja milloin se kannattaa
- [Claude Code: Plugins](#) — plugin-rakenne ja asennus

Spesifikaatiosivu `agent-plugins.org` ei tätä kirjoitettaessa vastaa oikealla TLS-sertifikaatilla (selain varoittaa), joten yllä on toimivat lähteet.

Miksi tätä repon ei ole paketoitu pluginiksi? Harkittiin ja jätettiin tekemättä kolmesta syystä:

1. Claude Coden plugin lukee skillit polusta `skills/` plugin-juuressa, ei polusta `.claude/skills/`. Skillit pitäisi siis joko siirtää, jolloin pelkkä repon kloonaukseen ei enää lataisi niitä, tai monistaa, jolloin kaksi kopiota eriytyy.
2. Plugin-formaatti on Claude Code -kohtainen. Se ei auta Copilotin, Cursorin tai Codexin käyttäjää, ja koko repon idea on että sama tiedosto käy kaikille.
3. `cp -r` ei ole kenellekään oikea este.

Jos jakelu joskus kasvaa, paketointi kannattaa tehdä silloin. Tilanne muuttuu jos työkaluriippumaton spesifikaatio vakiintuu niin, että yksi paketti palvelee kaikkia agentteja.

CSC:n omat dokumentit

- [Rahti](#) · [Satama](#) · [Allas](#)
- [Roihu](#) · [LUMI](#) · [Aitta](#)